PASCAL: A Phase-Aware Scheduling Algorithm for Serving Reasoning-based Large Language Models

Eunyeong Cho
KAIST
eunyeong.cho@kaist.ac.kr

Jehyeon Bang
KAIST
jehyeon.bang@kaist.ac.kr

Ranggi Hwang*
UNIST
ranggi.hwang@unist.ac.kr

Minsoo Rhu
KAIST
mrhu@kaist.ac.kr

2026 IEEE International Symposium on High Performance Computer Architecture (HPCA) | 979-8-3315-9302-5/26/$31.00 ©2026 IEEE | DOI: 10.1109/HPCA68181.2026.11408493

***Abstract*—The emergence of reasoning-based LLMs leveraging Chain-of-Thought (CoT) inference introduces new serving challenges, as their extended reasoning phases delay user-visible output and inflate Time-To-First-Token (TTFT). Existing LLM serving frameworks fail to distinguish between reasoning and answering phases, leading to performance degradation under GPU memory constraints. We present PASCAL, a phase-aware scheduling algorithm that prioritizes reasoning to reduce TTFT while using controlled preemption and token pacing during answering to preserve Quality-of-Experience (QoE). Our hierarchical scheduler combines instance-level placement with intra-instance execution and enables dynamic migration at phase boundaries to balance load and reduce interference. Across benchmarks using DeepSeek-R1-Distill-Qwen-32B, PASCAL reduces tail TTFT by up to 72% while maintaining answering phase SLO attainment, demonstrating the importance of phase-aware scheduling for reasoning-based LLM deployment.**

***Index Terms*—Serving system, reasoning-based LLMs, scheduling framework, user experience.**

I. INTRODUCTION

Large language models (LLMs) [5], [8], [15], [37] have emerged as powerful tools for various natural language processing tasks. As these models evolve, their deployment for real-time applications has become increasingly important, with a particular focus on inference efficiency and user experience. In LLM inference, user experience is significantly influenced by two key performance metrics. *Time-To-First-Token* (TTFT) measures the latency between the query submission time and the time the first response token is received. *Time-Per-Output-Token* (TPOT), on the other hand, measures the generation speed of each subsequent token. Prior work has established that these metrics correlate strongly with user satisfaction and engagement [2], [18], [22], [26], [31], [40], [54], making them critical design objectives when LLMs are deployed.

These user experience metrics are directly tied to the underlying computational stages of LLM inference. As shown in Figure 1(a), conventional LLM inference consists of two distinct stages: (1) the *prefill* stage processes the entire input prompt in a single forward pass and primarily determines TTFT, while (2) the *decoding* stage generates tokens auto-regressively, directly influencing TPOT. This clear distinction between stages has enabled prior works to develop targeted optimization strategies that align the prefill stage and decoding stage with user satisfaction metrics. Moreover, recent serving systems leverage the distinct computational characteristics of the prefill and decoding stages to better meet *service-level objectives (SLOs)* for TTFT and TPOT through disaggregated execution of prefill and decoding stage. [34], [39], [43], [54].

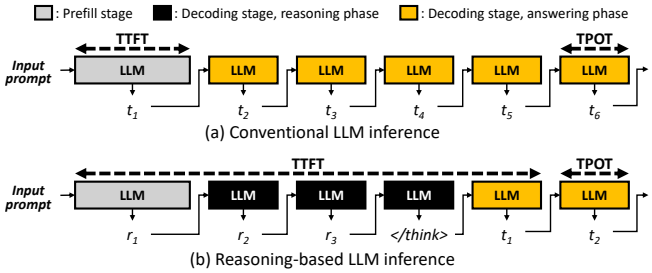

Fig. 1: Comparison of how TTFT and TPOT are defined in conventional LLMs versus reasoning-based LLMs. TTFT is the latency between the query submission time and the time the first "user-visible" response token (t_1) is received by the user. (a) In conventional LLMs, this first user-visible token is generated at the end of the prefill stage, so TTFT equals the prefill-stage latency. (b) In contrast, for reasoning-based LLMs, TTFT is the sum of the prefill-stage latency, the latency of the reasoning phase, and the latency from the end of the reasoning phase to the generation of the first answering token.

Although these approaches have been effective for conventional LLMs, the research landscape has evolved significantly with the emergence of *reasoning*-based LLMs [16], [28], [38], [45]. Reasoning models incorporate Chain-of-Thought (CoT) techniques [12], [46], [47], [49], [50], [55], which have gained tremendous popularity for their ability to solve complex problems by explicitly generating *intermediate* reasoning steps "before" producing final answers. We observe that this fundamental shift in how models generate outputs introduces new challenges for inference optimization, as detailed below.

Unlike conventional LLMs, reasoning-based LLMs produce intermediate *reasoning tokens* that represent step-by-step problem-solving, followed by *answering tokens* that convey the final outcome (Figure 1(b)). This distinction disrupts the conventional relationship between performance metrics (i.e., TTFT and TPOT) and the serving pipeline stages (i.e., prefill and decoding stages). Specifically, in reasoning-based LLMs, the decoding stage itself is divided into two functionally distinct phases: a *reasoning phase*, where reasoning tokens are generated, and an *answering phase* that produces the final answering tokens. The key insight here is that the performance and SLO requirements differ between these two phases. Because the user does not require direct insight into

*Work done while at KAIST.

the model's internal reasoning tokens, the top priority is to expedite the reasoning phase as fast as possible, reducing the time it takes to generate the model's reasoning tokens before the final answering tokens start streaming in. By minimizing the reasoning phase latency, the serving system can begin transmitting the user-visible tokens (i.e., the answering tokens) earlier, improving the user-perceived responsiveness by minimizing the latency to generate the first meaningful answering token. In contrast, the answering phase just has to be "*fast enough*" to ensure a smooth and prompt user experience, as it can tolerate somewhat looser throughput targets compared to the user-hidden reasoning phase. In other words, once the user begins seeing the answering tokens, maintaining a modestly high streaming rate is often sufficient for good user experience. Overall, TTFT in a reasoning model effectively encompasses not only the prefill stage but also the decoding stage of potentially numerous intermediate reasoning tokens (which are not necessary for the user to comprehend the final answer) and the latency from the end of the reasoning phase to the generation of the first answering token (Figure 1(b)).

To this end, we present PASCAL, a phase-aware scheduling algorithm that optimizes the distinct requirements of the reasoning and answering phases to enhance user experience while maintaining serving throughput. Our characterization of reasoning-based LLMs reveals the following key observations where (1) the absolute latency of the decoding stage, which directly impacts the reasoning model's TTFT, is highly sensitive to interruptions in the execution pipeline, while (2) its TPOT requirements can be maintained much more robustly with greater scheduling flexibility. Building on this insight, PASCAL introduces a phase-aware, hierarchical scheduling architecture. Our proposal explicitly partitions the decoding stage into reasoning and answering phases, applies distinct scheduling priorities to each, and enables instance-level load balancing to address the unique challenges of serving reasoning models. Through extensive evaluation on state-of-the-art reasoning-based LLMs, we show that PASCAL not only reduces the time to first answering token but also preserves or improves SLO satisfaction during the answering phase.

II. BACKGROUND

A. LLM Serving Metrics and Execution Phases

LLM serving systems are commonly evaluated using two critical performance metrics: *Time-To-First-Token* (TTFT), which measures how quickly a user receives the first generated token, and *Time-Per-Output-Token* (TPOT), which quantifies the generation speed once decoding begins. These metrics correspond to distinct points in the response timeline, aligning with the two computational stages of LLM inference: the prefill and decoding stages (Figure 1(a)).

During the prefill stage, the model processes the entire input prompt in a single pass, generating key-value (KV) representations (referred to as KV caches) for all tokens, and generating one output token (e.g., t_1 in Figure 1(a)). This stage is compute-intensive, exhibits high parallelism, and directly determines TTFT in conventional LLMs. The subsequent decoding stage auto-regressively generates output tokens by reusing the KV caches and incrementally updating them with each newly generated token. This stage is known to be memory-bandwidth-bound and determines TPOT. In practice, LLM serving systems define a *service-level objective (SLO)* to ensure a responsive user experience, typically targeting a TTFT of a few seconds or less and a TPOT of 5–10 tokens per second to align with human reading speeds [18], [31], [54].

B. Need for Blocking & Preempting Requests in LLM Serving

Modern LLM serving systems employ *continuous batching* [51], where new inference requests are incrementally added to the currently running batch at each decoding step. During decoding, however, the KV cache grows as new tokens are generated. Given limited GPU memory, the number of concurrent requests that can be batched is therefore constrained by the cumulative KV cache footprint [4], [7], [42], [52], [53]. Consequently, when GPU memory becomes saturated, the serving system must either *block* newly arriving requests or *preempt* (evict) in-flight requests to free GPU memory. Concretely, blocking occurs when an incoming request cannot be scheduled due to insufficient memory; it waits in the queue until enough KV cache space is released by completing requests. Because execution cannot begin until admission, this queuing delay inflates the request's TTFT. Preemption, in contrast, temporarily evicts an executing request; its KV cache is offloaded to CPU memory and execution is suspended. The request later resumes once sufficient GPU memory becomes available, reloading its KV cache to the GPU. Since preemption halts token generation, it increases TPOT.

C. LLM Scheduling Policy

An LLM serving system's scheduling policy determines which requests' KV caches can remain resident in GPU memory and which should be blocked or preempted (and thus offloaded) to preserve GPU memory space. Early systems such as `vLLM` [27] rely on a simple *First-Come-First-Served* (FCFS) scheduling policy that batches requests in arrival order. When GPU memory is exhausted under FCFS, the most recently arrived requests are preempted by offloading their KV caches to CPU memory. FCFS then blocks incoming requests until GPU memory becomes available, and preempted requests are resumed after their KV caches are reloaded from CPU to GPU memory. This mechanism significantly impacts TTFT, as new requests must wait for memory to free up, suffering from Head-of-Line (HoL) blocking where earlier long-running requests delay the servicing of later ones. Figure 2(a) illustrates an ideal scenario without GPU memory constraints, where Request C can be immediately batched with earlier requests. In contrast, Figure 2(b) shows a case where Request C must wait for Requests A and B to complete, resulting in an excessive TTFT of 7 time units. This is particularly problematic, as modern LLM serving systems define SLOs for both TTFT and TPOT to ensure responsiveness. FCFS inherently increases the

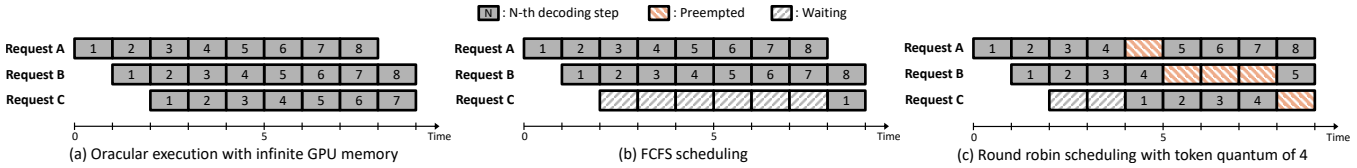

Fig. 2: Timeline illustrating how the serving system handles requests under three different scenarios. We assume that Request A, B, and C arrive at times 0, 1, and 2, respectively, and that the GPU memory capacity allows at most two requests to be batched simultaneously. (a) With oracular execution under infinite GPU memory, there is no limit on how many requests can be batched, so each request begins execution immediately upon arrival, and no preemption occurs. (b) Under FCFS scheduling, the first two requests (A and B) are batched, while C must wait. Request C joins the batch only after A finishes decoding, yielding a TTFT of 7 time units. (c) In round-robin (RR) scheduling, each request decodes a fixed quantum of four tokens before being preempted. Request C waits for 2 time units, joins the batch when Request A is preempted at time 4, and is itself preempted after producing four tokens at time 8.

risk of violating TTFT objectives due to HoL blocking under GPU memory pressure.

To address this limitation, recent work proposes various SLO-aware schedulers [14], [26], [31], [48]. One line of this research explores *time-sharing* within a single serving instance. In this context, a serving instance is the software abstraction in LLM serving frameworks that serves as the unit of execution and manages a replica of the model weights, coordinating access to shared GPU resources. Time-sharing schedulers [26], [31], [48] utilize preemption to allocate GPU time slices to concurrent requests, preventing any single long-running request from monopolizing the GPU memory. In addition to mitigating HoL blocking and improving TTFT, these approaches also help maintain acceptable TPOT by enabling fair resource allocation during decoding. A canonical example of time-sharing scheduler is the classical *round-robin* (RR) algorithm. In this approach, the scheduler assigns each request a fixed *token quantum*. Once a request consumes all its assigned quantum, its scheduling priority is lowered. If the GPU memory becomes limited, the scheduler can preempt the lowest-priority requests by offloading their KV caches to CPU memory. This frees up memory, allowing newly arrived requests to be admitted into the batch and begin decoding, thereby reducing their TTFT. More advanced schedulers are based on the same key insight, with priorities determined by different heuristics, for example, the likelihood of SLO violation [31], the number of generated tokens per request [26], or estimated completion deadlines [48]. As shown in Figure 2(c), the earliest request, Request A, is preempted by the scheduler after generating a predefined token quantum (4 tokens in this example). This frees sufficient memory for Request C to join the batch and generate its first token within 3 time units.

An unfortunate side effect of time-sharing schedulers is that they can cause irregular token-generation rates. In Figure 2(c), for instance, Request B is preempted for 3 time units, increasing the latency between its fourth and fifth tokens and potentially degrading user experience. Prior work [31] addresses this issue using a *token pacer*. The token pacer temporarily buffers tokens when they are generated in bursts and tries to release them at a steady rate whenever the request is idle due to preemption, thereby smoothing the output rate perceived by the user. To quantitatively measure how well

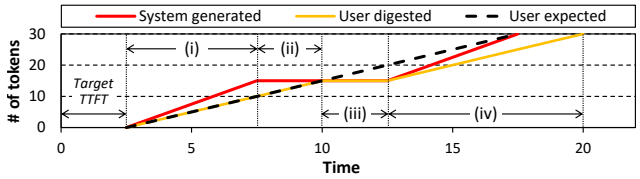

Fig. 3: An example token serving scenario with QoE measurement, where token delivery begins at the target TTFT without additional delay. (i) The serving system initially generates tokens (red line) faster than the user's expected reading pace (black dotted line), causing tokens to be buffered by the token pacer. The user consumes tokens at their own pace, represented by the user digested line (yellow). (ii) When the serving system is temporarily paused, the user continues consuming the buffered tokens. (iii) Once all the tokens kept in the buffer are depleted, the user experiences starvation until token generation resumes. (iv) Once token generation resumes, the user continues consuming tokens at a steady pace, although the user's token digestion timeline remains behind the expected schedule due to the earlier delay experienced when the serving system paused. QoE is measured by computing the ratio between the area under the user digested token curve (yellow) and the area under the user expected curve (black dotted line).

the serving system's token generation rate aligns with the user's token consumption rate, [31] introduced the *Quality-of-Experience (QoE)* metric, which more accurately reflects SLO attainment than the conventional average TPOT measure. QoE combines two factors: (1) whether each request meets the target TTFT, and (2) how closely the effective token generation speed delivered to the user matches the target TPOT, accounting for the effects of token pacing. Figure 3 illustrates how QoE is measured, which reflects how closely the token generation rate aligns with the user's expected responsiveness. QoE is normalized to the interval [0,1] and a score of 1 indicates perfect alignment, while lower values suggest delays or starvation in token delivery (e.g., the example in Figure 3 exhibits a QoE value less than 1 due to the lagging of token digestion timeline vs. the user expected timeline).

D. Rethinking TTFT and TPOT in Reasoning-Based LLMs

Reasoning-based LLMs generate more accurate responses and solve complex problems by incorporating techniques like Chain-of-Thought (CoT) [46], [47], [50], [55], in which models explicitly work through intermediate reasoning steps before arriving at final answers. This approach mimics human

problem-solving by breaking down complex tasks into logical sequences. Over time, several reasoning algorithms have emerged, ranging from structured approaches such as tree-structured reasoning [50] to explicit prompting methods [47]. More recently, the field has converged toward methods in which the reasoning process is fully integrated into a single decoding pipeline, exemplified by models such as DeepSeek-R1 [16] and OpenAI o3 [38]. These integrated reasoning-based LLMs automatically perform internal deliberation steps (i.e., the reasoning phase) within the same neural network architecture and inference process. They employ advanced training methods that enable the internalization of reasoning capabilities directly within the model architecture [16], [38].

These fundamental differences from conventional LLMs affect how serving metrics should be interpreted. In conventional LLMs, TTFT and TPOT map cleanly to two distinct execution stages: TTFT captures the latency of the prefill stage, while TPOT measures how quickly output tokens are generated during the subsequent decoding stage. As illustrated in Figure 1(a), conventional LLMs generate the first user-visible token (t_1) immediately after the prefill stage, so TTFT spans only the grey-colored block. In contrast, reasoning-based LLMs internally divide the decoding stage into two sub-stages: (1) a *reasoning phase*, which generates latent reasoning tokens not visible to the user, and (2) an *answering phase*, producing the user-visible answering tokens. As shown in Figure 1(b), a reasoning-based LLM first generates reasoning tokens (r_1, r_2, and r_3) before the answering phase begins at t_1. As a result, the latency to generate these reasoning tokens contributes to TTFT, even though they occur during decoding. That is, TTFT of reasoning-based LLMs consists of sum of the prefill-stage latency, the latency to execute the reasoning phase, and also the latency from the end of the reasoning phase to the generation of the first answering token (Figure 1(b)). Consequently, the decoding stage now contributes to both TTFT (during the reasoning phase) and TPOT (during the answering phase), breaking the conventional one-to-one mapping of TTFT to prefill and TPOT to decode. For the remainder of this paper, we define the reasoning phase as encompassing both the prefill stage and the generation of reasoning tokens within the decoding stage, as both contribute to the latency before the first answering token becomes visible to the user.

III. Characterization and Motivation

A. Methodology

This section characterizes how different scheduling policies impact reasoning-based LLM inference. We present key insights that inform efficient scheduling decisions for reasoning-based LLMs, with a particular focus on optimizing user experience. All experiments in this section were conducted on a server node hosted by Intel Xeon Platinum 8558 CPU (with 256 GB DDR5 DIMM) communicating with an NVIDIA H100 GPU (with 96 GB HBM) communicating over PCIe 5.0. For LLM serving, we employ vLLM v0.6.1 [27].

Workload. We conducted experiments using DeepSeek-R1-Distill-Qwen-32B [9], [16], an open-source LLM designed

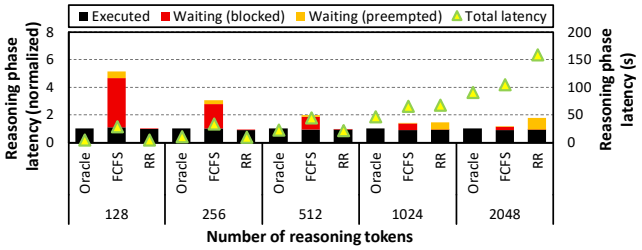

Fig. 4: (Left axis) Breakdown of average reasoning phase latency across oracle, FCFS, and RR scheduling policies for varying numbers of reasoning tokens (x-axis), normalized to the oracle latency at each reasoning token count. Executed (black) denotes time the request actively ran on the GPU without being blocked (red) or preempted (yellow). (Right axis) Corresponding average absolute latency in seconds, indicated by green triangles.

for reasoning tasks. To clearly characterize and motivate how different scheduling decisions affect the SLO metrics of the reasoning-based LLMs, we construct synthetic workloads as follows. Two separate experiments are conducted independently, each focusing on either the reasoning phase (Figure 4) or the answering phase (Figure 5). In each experiment, our LLM serving system processes a total of 300 requests, with request arrivals following a Poisson distribution. The first experiment targets the reasoning phase (Figure 4) where all requests have a fixed input prompt length of 128 tokens, while the reasoning token length is randomly selected between 128 and 2048 tokens. This setup allows us to isolate how different scheduling policies impact the performance of the reasoning phase under varying reasoning token lengths. In the second experiment characterizing the answering phase (Figure 5), we assume that all requests have already completed the execution of both prefill and the reasoning phase (the combined length of prefill and reasoning tokens is fixed at 128 tokens), and that their corresponding KV caches are already generated during previous execution. Under this setting, each request generates answering tokens with lengths ranging from 128 to 2048, which is chosen randomly, allowing us to analyze how different scheduling policies affect SLO attainment during the answering phase.

Scheduling policy. To demonstrate how GPU memory constraints influence scheduling efficacy, we compare two scenarios: (1) an *oracle* configuration with sufficient memory to hold the KV caches of all 300 requests simultaneously, allowing the scheduler to never have to block or preempt requests due to GPU memory limits, and (2) a memory-constrained configuration. In our NVIDIA H100 (96 GB) GPU based system, the full-capacity setting can accommodate all KV caches concurrently, so we treat it as the oracle configuration with no performance degradation attributable to memory limits. To emulate a realistic serving environment with GPU memory constraints, we cap the GPU memory available for KV cache allocation to 50% of the oracle capacity and schedule requests based on FCFS or RR, which will block incoming requests or preempt ongoing ones once memory is exhausted (Figure 2).

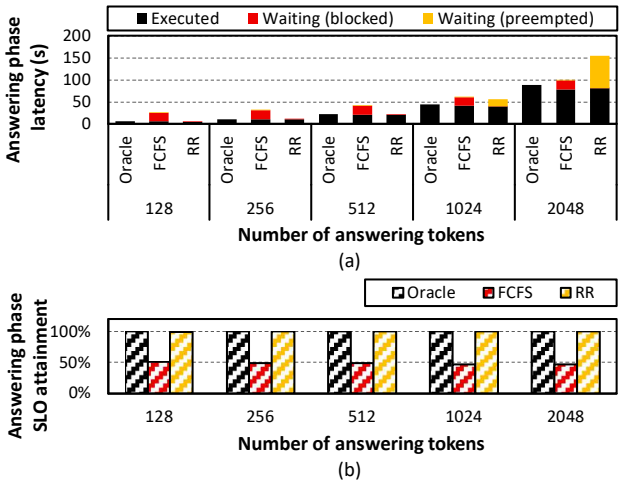

Fig. 5: (a) Answering phase latency breakdown across varying numbers of answering tokens and (b) the corresponding SLO attainment rate. The SLO target for the answering phase is determined by two factors: (1) the latency between the end of the reasoning phase and the generation of the first answering token (henceforth referred to as *Time-To-First-Answering-Token* (TTFAT)), and (2) whether the answering token generation rate meets the TPOT target. As such, per our definition of QoE in Figure 3, the answering phase's QoE is determined by the target values of TTFAT and TPOT. Following prior work [54], we set the target TTFAT and TPOT values to 0.25 seconds and 100 milliseconds, respectively. A request violates the SLO if its QoE score falls below 0.95.

B. Characterization

Reasoning phase. Figure 4 illustrates how blocking and preemption aggravate reasoning phase latency relative to the oracle scenario (Figure 2(a)). Unlike conventional LLMs where TTFT is determined solely by the prefill stage, reasoning-based LLMs produce the first (user-visible) answering token only after *all* reasoning tokens are generated. Consequently, TTFT spans not only the prefill stage but also the entire decoding of reasoning tokens and can far exceed the sub-second TTFT SLO typical of conventional LLMs, making timely completion of the reasoning phase critical for user satisfaction. Figure 4 therefore focuses on how scheduling policies shape reasoning phase latency under memory sufficiency (oracle) vs. memory pressure (FCFS, RR). As shown, both blocking (FCFS) and preemption (RR) add directly to reasoning phase latency, especially when limited GPU memory prevents uninterrupted execution. Under FCFS, blocking inflates latency because a request must wait behind long-running ones whose KV caches remain resident until completion. This effect is most pronounced for short reasoning requests (e.g., 128 reasoning tokens), where wait time dominates and latency increases by up to $5.14\times$ over the oracle. In contrast, RR avoids outright blocking by interleaving requests, but frequent preemptions fragment execution. For long reasoning requests (e.g., 2048 reasoning tokens), RR increases latency by up to $1.75\times$ vs. uninterrupted oracle execution. Because RR enforces a fixed token quantum, long requests cannot retain continuous residency of their KV caches in GPU memory, accumulating repeated interruption and waiting intervals. Similar degradation is expected for other time-sharing policies that necessarily preempt long requests to preserve fairness.

In summary, both blocking (FCFS) and preemption (RR and other time-sharing schedulers) during the reasoning phase substantially delay generation of the first answering token under GPU memory constraints, severely degrading TTFT and motivating memory- and phase-aware scheduling strategies.

Answering phase. Figure 5 shows how scheduling policies affect user experience during the answering phase. Figure 5(a) shows a breakdown of the latency of answering phase across varying numbers of answering tokens and Figure 5(b) shows the corresponding SLO attainment rate. In reasoning-based LLMs, TTFT is measured as the latency until the "first" answering token is generated. Unlike the reasoning phase, where any additional delay directly increases TTFT, the answering phase can still meet its SLO if (1) the latency from the end of reasoning to the first answering token does not exceed the *Time-To-First-Answering-Token* (TTFAT) target, which is set to be near-instantaneous to ensure an immediate transition from the reasoning phase to the answering phase, and (2) the steady-state answering token generation rate meets the TPOT target (Figure 3)[1]. Because SLO satisfaction is threshold-based rather than minimizing absolute latency (i.e., latencies only need to be *fast enough*), appropriate scheduling can preserve user experience even when execution is fragmented by blocking or preemption.

Under FCFS, blocking forces requests to wait behind long-running ones (Figure 5(b)), substantially increasing TTFAT and lowering SLO attainment across all answering token lengths (Figure 5(b)). In contrast, RR eliminates head-of-line blocking by interleaving execution, allowing all requests to start promptly to keep TTFAT low, while also sustaining adequate token generation rate. Consequently, SLO attainment remains high for most configurations with RR. Notably, at 2048 answering tokens, RR incurs higher total answering phase latency than FCFS due to preemptions (Figure 5(a)), yet achieves the same SLO attainment as the oracle because both its TTFAT and per-token generation rate remain within thresholds. This shows that during the answering phase, time-sharing policies like RR can tolerate preemption overhead while still preserving user experience, unlike their detrimental impact during the reasoning phase, as highlighted in Figure 4.

In summary, answering phase SLOs are robust to moderate RR-induced preemption overhead as long as threshold metrics are met, making time-sharing schedulers like RR preferable to blocking policies like FCFS for this phase.

[1]Our target TPOT was selected from a user-centric perspective rather than being tied to GPU performance or model specifications. This decision is grounded in the principle that user satisfaction is largely determined by whether the LLM produces output tokens at a rate comparable to typical human reading speeds. In other words, even if future GPUs provide $10\times$ greater compute capability, we do not expect TPOT to decrease proportionally, as there is little benefit in "over-optimizing" it beyond the threshold of human perception. Likewise, even with future LLMs that demand $10\times$ more computation, we expect TPOT to remain within the 100 ms range. Prior studies [18], [22], [31], [32] have similarly argued that a TPOT of around 100 ms (which is already faster than human reading speeds) is sufficient to provide a natural, conversational experience.

C. Motivation

Our analysis reveals that reasoning and answering respond very differently to blocking and preemption. In the reasoning phase, blocking or preemption directly stretches reasoning latency, delaying the generation of the first answering token and degrading TTFT-driven user experience. In contrast, limited preemption does not necessarily harm user experience in the answering phase. As long as the first answering token appears promptly after reasoning phase's completion and the steady-state answering token generation rate meets the required throughput threshold, SLOs remain satisfied even if execution is fragmented. Time-sharing policies (e.g., RR) therefore mitigate head-of-line blocking and preserve answering-phase QoE, despite incurring preemption overhead.

In summary, two key insights emerge. (1) The reasoning and answering phases exhibit fundamentally different sensitivities: reasoning latency is highly interruption-sensitive, while answering QoE is threshold- (not absolute latency-) sensitive, and (2) time-sharing is especially effective in the answering phase because it avoids blocking without violating SLO thresholds.

Existing scheduling policies targeting conventional (single-phase) LLM inference ignore the distinct latency vs. threshold sensitivities of reasoning-based workloads. This gap motivates phase-aware scheduling that adapts its strategy, minimizing interruptions during reasoning while employing fair time-sharing during answering to jointly optimize reasoning latency and answering phase user experience.

IV. PASCAL: A Phase-Aware Scheduling Algorithm for Serving Reasoning-based LLMs

PASCAL is a scheduling system that improves the user experience for reasoning-based LLMs on multi-instance LLM serving systems. An instance is the software abstraction in LLM serving frameworks that serves as the unit of execution, managing a replica of the model weights and coordinating access to shared GPU resources. PASCAL schedules inference requests differently in the reasoning and answering phases by exploiting their asymmetric sensitivity to preemption. Specifically, it minimizes interruptions during the reasoning phase to reduce TTFT and permits controlled preemption during the answering phase without compromising SLO goals for TPOT.

A. High-level Overview

We take a top-down approach to explain PASCAL's two-level scheduling architecture (Figure 6). The *instance-level scheduler* selects which GPU instance should service each incoming request. The *intra-instance scheduler* then orchestrates request batching and kernel execution within the instance.

The instance-level scheduler employs two placement algorithms, one for each phase of a request: reasoning and answering. When a request first arrives, it enters the reasoning phase[2]. During this phase, the request generates intermediate reasoning tokens from the input prompt until reasoning completes. For requests in this phase, the scheduler selects the

[2]For clarity, we collectively refer to the prefill stage and the subsequent reasoning phase of decoding as the reasoning phase.

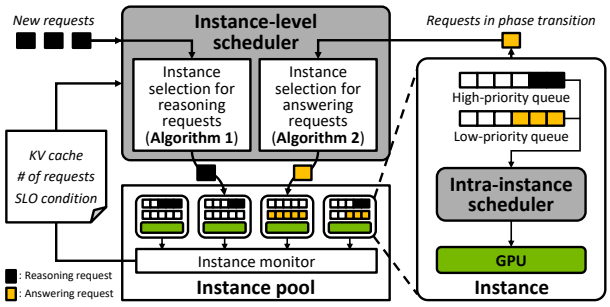

Fig. 6: Overview of PASCAL. The system comprises an instance-level scheduler and a pool of serving instances. All incoming requests first enter the reasoning phase and are routed to the appropriate instances using Algorithm 1. After a request transitions to the answering phase, the instance-level scheduler selects a (possibly different) instance using Algorithm 2. Within each serving instance, an intra-instance scheduler manages routed requests through high- and low-priority queues and orchestrates their execution. An instance monitor continuously collects runtime metrics to guide these placement decisions.

instance that minimizes reasoning latency while preserving the performance of other requests already executing on each GPU instance. After the system's instance monitor detects that the most recent token generated by an existing request is the special token signaling the end of the reasoning phase (e.g., the <\think> token in DeepSeek-R1), the scheduler re-examines whether migrating that request to a different instance would be beneficial. Such migration aims to minimize any delay after reasoning completes, satisfy answering-phase SLOs, and reduce interference with co-located requests.

Based on the instance-level scheduler's decisions, a single GPU instance may host requests from both the reasoning and answering phases. To manage these mixed phase requests effectively, PASCAL deploys a local intra-instance scheduler that coordinates execution across phases. It prioritizes the scheduling of reasoning requests by placing them in a high-priority scheduling queue, since their latency directly affects TTFT and is highly sensitive to blocking and preemption. Answering requests are inserted to a low-priority queue, and the scheduler time-shares the remaining GPU resources among them and seeks to preserve a consistent user experience. Requests within the same (high-/low-priority) queue are scheduled based on a round-robin policy.

B. Instance-level Scheduler

The instance-level scheduler selects the GPU instance for each request based on its current phase and SLO requirements. Consequently, the scheduler uses two distinct instance-selection algorithms, one per phase, each conditioned on the current status of an instance's request queues. As noted earlier, reasoning requests are routed to the high-priority queue within a GPU instance, whereas answering requests are routed to the low-priority queue. Accordingly, PASCAL employs two algorithms: one for assigning new reasoning requests (Algorithm 1) and another for determining whether requests that have entered the answering phase should be migrated to a different instance (Algorithm 2). These algorithms consider each instance's queue occupancy and the characteristics of

Algorithm 1 Instance Selection for Reasoning Requests

1: **Input:** Set of instances I, where for each instance $i \in I$, we have:
   t_i: whether all answering requests meet their SLOs (TRUE/FALSE)
   m_i: total memory occupied by KV cache (GPU + CPU)
2: **Output:** Selected instance to schedule the reasoning phase request
3: $E \leftarrow \{\, i \in I \mid t_i = \text{TRUE} \,\}$
4: **if** $E = \emptyset$ **then**
5: $\quad E \leftarrow I$
6: **end if**
7: $selected \leftarrow \text{argmin}_{i \in E}(m_i)$
8: **return** $selected$

Algorithm 2 Instance Selection for Answering Requests

1: **Input:** Set of instances I, where for each instance $i \in I$, we have:
   t_i: whether all answering requests meet their SLOs (TRUE/FALSE)
   r_i: number of reasoning requests in high-priority queue
   a_i: number of answering requests that has not exhausted the first time quantum
2: **Output:** Selected instance to schedule the answering phase request
3: $E \leftarrow \{\, i \in I \mid t_i = \text{TRUE} \,\}$
4: **if** $E = \emptyset$ **then**
5: $\quad E \leftarrow I$
6: $\quad$ **for all** $i \in E$ **do**
7: $\quad\quad r_i \leftarrow r_i + a_i$
8: $\quad$ **end for**
9: **end if**
10: $selected \leftarrow \text{argmin}_{i \in E}(r_i)$
11: **return** $selected$

individual requests, aiming to balance memory usage while respecting the phases' differing latency sensitivities.

Instance selection for reasoning requests. As described in Algorithm 1, the instance-level scheduler first inspects each GPU instance's token pacer to determine whether answering requests are being served at the expected user-perceived rate. If the token pacer reports insufficient remaining tokens (indicating token generation is slower than the user's expected pace), the instance is deemed to be violating its answering-phase SLO and is excluded from the candidate set (i.e., t_i = FALSE in line 3). Recall that answering requests are assigned a lower priority than the reasoning requests and thus must rely on whatever GPU memory remains once the high-priority queue has allocated KV cache space for reasoning requests. Therefore, an instance already missing its answering-phase SLO implies that available GPU memory is insufficient to support the current answering requests. Consequently, scheduling additional high-priority reasoning requests to such instance would only intensify memory pressure and further delay answering requests. Among the remaining SLO-compliant candidate instances, the scheduler selects the instance with the smallest KV cache footprint m_i (line 7). A smaller KV cache footprint generally indicates fewer active requests, making the instance more amenable to accommodating additional new requests without interfering with requests already under execution. Moreover, smaller KV cache correlates with shorter attention-layer execution time, increasing the likelihood of meeting TTFT for new reasoning requests, further justifying this design decision. When no instance meets the SLO condition (i.e., $E = \emptyset$ in Algorithm 1), the scheduler still chooses the instance with the minimum m_i to minimize additional performance degradation for in-flight requests (lines 4–5).

Instance selection for answering requests. During the execution of a reasoning request, the instance monitor in Figure 6 continuously checks for the special token (e.g., <\think>) that marks the transition to the answering phase. When this token appears (signaling that the model is ready to emit answer tokens), the instance-level scheduler migrates the request to a new GPU instance according to Algorithm 2, unless our adaptive migration policy (discussed further at the later part of this subsection) overrides such migration decision. As with Algorithm 1, any instance that is currently failing to meet answering-phase SLOs (i.e., t_i = FALSE) is removed from the candidate set (line 3). From the remaining candidates, the scheduler selects the instance with the fewest active reasoning-phase requests (r_i) to minimize the chance that the answer request is delayed by higher-priority reasoning requests (line 10). Because reasoning requests reside in the high-priority queue and allocate GPU memory first, an instance hosting many such requests often leaves too little GPU memory for a new answering request. Insufficient GPU memory forces answering requests either to spill their KV cache to CPU memory or to wait until resources are freed—both of which compromise answering-phase SLOs. This motivates selecting the instance with the smallest r_i for answering requests.

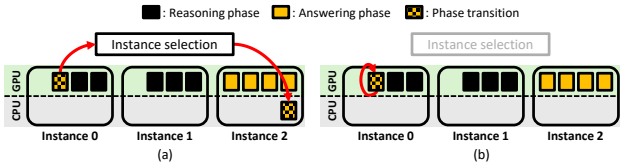

Fig. 7: Adaptive migration example. (a) Algorithm 2 initially recommends migrating the phase transitioning request to Instance 2. Because GPU memory in Instance 2 is fully utilized (indicated by four yellow blocks) while Instance 0 still has free memory (two empty slots), (b) the system overrides Algorithm 2's migration decision and continues executing the phase transitioning request in Instance 0.

If no instance satisfies the SLO condition, the scheduler selects the one with the smallest $r_i + a_i$ (lines 4–9), where a_i denotes the number of answering requests in the low-priority queue that have not yet exhausted the first time quantum (i.e., they are more likely to be scheduled next under the RR policy). As described in Section II, RR scheduling is fairness-oriented; the more "fresh" answering requests (a_i) an instance holds, the more the newly arriving answering request must compete for scheduling opportunities, an undesirable outcome when selecting the instance for new answering requests. Our empirical evaluation confirms that PASCAL's such heuristic based instance-selection algorithm, which considers both r_i and a_i, achieves better load balancing and SLO attainment than using r_i alone under these scenarios.

Adaptive migration. Although requests are generally migrated to a new instance when they transition to the answering phase, strictly following Algorithm 2's decision can lead to suboptimal resource utilization in certain scenarios. Therefore, PASCAL introduces an *adaptive* migration mechanism that determines whether to migrate a request during phase transitions based on the current memory availability. This

strategy becomes particularly important in scenarios where reasoning and answering requests are distributed asymmetrically across instances. Figure 7(a) shows one such case when reasoning requests are distributed across many instances, while answering requests are concentrated on a certain instance. Since answering requests are directed to instances with the fewest active reasoning requests (Algorithm 2), they tend to accumulate on the same instance (`Instance 2`). In this case, when a reasoning request in `Instance 0` transitions to the answering phase, Algorithm 2 likely selects `Instance 2` based on the number of reasoning requests, which lacks available GPU memory. Despite `Instance 0` having sufficient memory to handle the answering phase, migrating the request to `Instance 2` can cause the unnecessary execution stall for answering requests in `Instance 2`. To handle this case, at phase transition, if the current instance has sufficient GPU memory while the one selected for answering phase does not, PASCAL preserves the request on the current instance rather than migrating it to the selected one (Figure 7(b)). This avoids unnecessary CPU offloading and KV cache transfer overhead, and it better utilizes available GPU memory.

KV cache transfer overhead. Migrating a request at phase transition requires transferring its entire KV cache from the source to the destination instance. Unlike prior work [39], [44], [54], which can overlap KV cache transfers with computation, reasoning-based LLMs cannot predict phase transitions in advance, as transitions only become apparent when the model emits specific tokens (e.g., <\think>) that indicate the end of the reasoning phase. In reasoning-based LLMs, this transfer latency can impact TTFT because the server must wait for both the completion of the reasoning phase and the subsequent KV cache migration *before* generating the first answering token. Nonetheless, we find that the overhead of KV cache transfers is negligible in reasoning models, as TTFT is primarily dominated by reasoning phase latency. For example, Patel et al. [39] analyzed KV cache transfer latency in a disaggregated LLM serving system and reported a one-time transfer delay of approximately 40ms for 2,048 tokens. To put this in context, consider a per-token latency of 30ms [39], which reflects an aggressive decoding speed. Because reasoning phases can involve hundreds or even thousands of tokens—equating to several tens of seconds of reasoning latency—the one-time KV cache transfer constitutes only a small fraction of the total TTFT and is therefore negligible.

C. Intra-instance Scheduler

This subsection explains how the intra-instance scheduler manages and executes requests within each GPU instance. As shown in Figure 6, each instance maintains a hierarchical priority queue that separates the reasoning and answering phases based on their distinct performance requirements:

- **High-priority queue** stores requests in the reasoning phase. These requests are served first and receive preferential allocation privilege to GPU memory, minimizing the impact of blocking or preemption on TTFT.

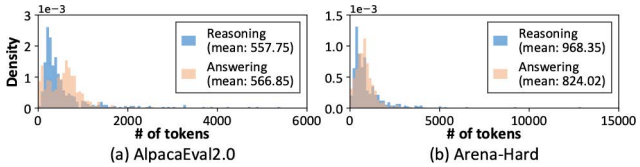

Fig. 8: Reasoning and answering token count distribution for (a) AlpacaEval2.0 [10] and (b) Arena-Hard [29] datasets.

- **Low-priority queue**: stores requests in the answering phase. Because these requests tolerate preemption better, they are executed using whatever GPU memory remains after high-priority allocations are satisfied.

Occasionally, a single reasoning request with an exceptionally long sequence and a very large KV cache can exhaust the memory needed by answering requests. In such cases, the intra-instance scheduler *demotes* the reasoning request whose KV-cache size exceeds a user-defined threshold to the low-priority queue, effectively treating it as a low-priority answering request. This conditional demotion policy frees GPU memory for answering requests more quickly while minimally impacting reasoning-phase SLOs.

For each priority queue, the scheduler applies a different dispatch strategy. For reasoning requests in the high-priority queue, PASCAL adopts a RR policy. As discussed in Figure 4, when the serving system is constrained by GPU memory, offloading even high-priority reasoning requests becomes inevitable, and both FCFS and RR experience latency degradation. However, the severity of this degradation differs: FCFS increases average latency due to head-of-line blocking across all request lengths, whereas RR provides much faster responsiveness for short reasoning requests at the cost of higher tail latency (Figure 4). To this end, PASCAL selects RR for the high-priority queue to guarantee lower TTFT for short reasoning requests under memory pressure.

Unlike reasoning requests, tokens generated during answering requests are streamed directly to users, making perceived responsiveness critical. As evaluated in Figure 5, RR exhibits high robustness to preemption-induced latency overheads and achieves high SLO attainment rates. To mitigate user-experience degradation under preemption or delayed execution, PASCAL augments RR with a token pacer for answering requests in the low-priority queue. The token pacer regulates the token emission rate to match user expectations, allowing even preempted answering requests to appear responsive. This mechanism enables PASCAL to maintain a high answering-phase SLO success rate under constrained memory, outperforming an FCFS baseline.

V. EVALUATION

A. Methodology

Simulation framework. We developed a cluster-level simulator to evaluate design points enabled by PASCAL. Inspired by the methodology in [39], our simulator models eight server nodes (instances) connected via a 100 Gbps fabric, each with an NVIDIA H100 96 GB GPU. For single serving instance simulation, we adopt a *profile-based* LLM serving

methodology, which prior work has shown to strike a balance between simulation speed and accuracy [1], [6], [30], [39]. The single-instance simulator uses vLLM profiling data and mirrors established techniques [1], [6], [30], [39] to capture GPU execution behavior. We validate it by comparing simulated and measured inference latency on a real system with an Intel Xeon Platinum 8558 CPU and H100 GPU (PCIe 5.0, vLLM 0.6.1, PyTorch 2.4.0, CUDA 12.1). Our simulator achieves a mean absolute percentage error (MAPE) of 1.62% for end-to-end latency and captures key metrics with MAPE values of 12.6% for mean TTFT and 6.49% for TPOT.

Model and dataset. We evaluate PASCAL using DeepSeek-R1-Distill-Qwen-32B [9], [16], an open-source reasoning LLM. The 32B model was chosen to reflect LLM serving scenarios in which GPU memory constraints limit the allocation of KV caches across diverse requests, forcing the scheduler to block or preempt requests as necessary. Serving traces are built from AlpacaEval2.0 [10] and Arena-Hard [29] prompts, queried via OpenAI's o4-mini [38] API, which provides reasoning/answering token counts (Figure 8). These two benchmarks, also used in DeepSeek-R1's evaluation [16], capture chat applications requiring detailed responses. We additionally analyze scenarios with long reasoning but short answers (e.g., problem-solving) in Section V-D.

Scheduling policy. We compare PASCAL against two baseline scheduling algorithms as follows:

- **FCFS**: The default policy in vLLM. Requests are served strictly in arrival order without distinguishing reasoning and answering phases. When KV cache usage exceeds GPU memory, the KV cache spill to CPU memory, and new admissions pause until space frees up.
- **Round-robin (RR)**: A preemptive scheduler that allocates fixed token quanta to active requests in circular order, regardless of which phase the request is executing; requests that finish their quantum are deprioritized.

For both baselines, reasoning requests are placed on the instance with the smallest KV footprint, and no migration occurs at phase transitions. Token quantum is set to 500 for RR and for each queue in PASCAL. In PASCAL, reasoning requests exceeding 5000 tokens are demoted to the low-priority queue.

Metric. We evaluate schedulers using two metrics: *TTFT* and *SLO violation*. TTFT measures latency from request submission to the first answering token. SLO violations are assessed via QoE, calculated from TPOT starting at the first answering token. Because reasoning-based LLMs have highly variable reasoning lengths, the original QoE metric (which includes a fixed TTFT target) is impractical; we instead compute QoE solely from TPOT and evaluate TTFT separately. An SLO violation is counted when QoE falls below 0.95.

B. User Experience (TTFT, SLO Violation) and Throughput

TTFT. Figure 9 shows the raw TTFT values of all requests as a function of reasoning token length and Figure 10 summarizes the resulting *tail* TTFT. Below we focus on tail TTFT because most requests fit within GPU memory and

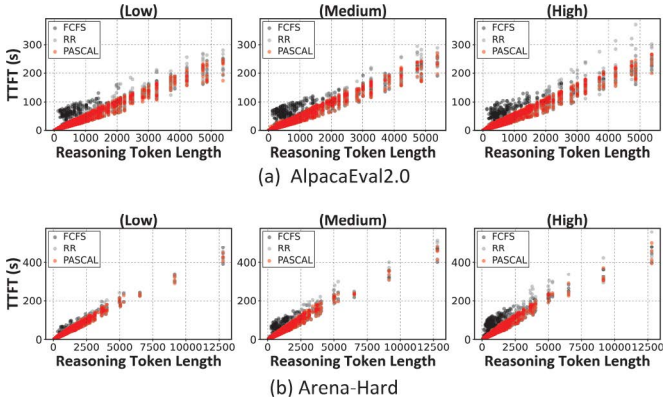

Fig. 9: Absolute TTFT for all scheduled requests as a function of reasoning-token length. Experiments were conducted under three request-arrival rates (low, medium, and high). The "high" rate stresses the LLM serving system more severely; exceeding GPU compute and memory capacity increases the likelihood of preemption and blocking. Request arrivals follow a Poisson distribution.

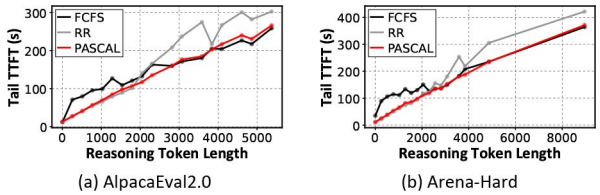

Fig. 10: Tail TTFT latency as a function of reasoning token length under high request-arrival rates. Requests are grouped into 256-token bins based on reasoning token length (e.g., [0–255] bin, [256–511] bin, . . .). These workloads exhibit a highly skewed distribution where more than 70% of requests generate fewer than 1,000 reasoning tokens. This skew causes substantial variation in the number of requests that fall under a given bin. Consequently, we use different tail latency metrics based on the sample size of each bin: maximum TTFT is used for bins with fewer than 10 samples, 90th-percentile (P90) for bins with fewer than 20 samples, P95 for bins with fewer than 100 samples, and P99 otherwise. Bins with fewer than five samples are omitted as we believe they are statistically less meaningful.

scheduling effects are most pronounced in the tail, where memory pressure triggers cache spills and preemption.

The baseline FCFS suffers from head-of-line blocking because once a request starts reasoning, it holds GPU memory until its answering phase completes. As a result, even short-reasoning requests experience severe tail TTFT degradation. RR can mitigate head-of-line blocking, which improves TTFT for shorter requests. In RR, answering phases always follow reasoning phases within each request's lifecycle, but they receive progressively lower priority as the request consumes more time quanta. This behavior implicitly creates a per-request hierarchy (i.e., scheduling priority) that favors reasoning, similar to PASCAL. However, RR cannot enforce a global "reasoning-first" scheduling priority as it does not distinguish the reasoning phase vs. answering phase. For example, a short request that quickly enters its answering phase can retain higher priority than a long reasoning request that has already consumed many quanta, rendering RR's fairness-oriented policy to deprioritize the longer re-

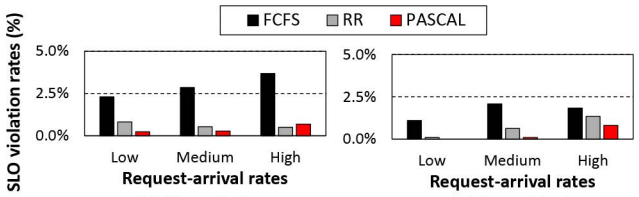

Fig. 11: SLO violation rates across different request-arrival rates.

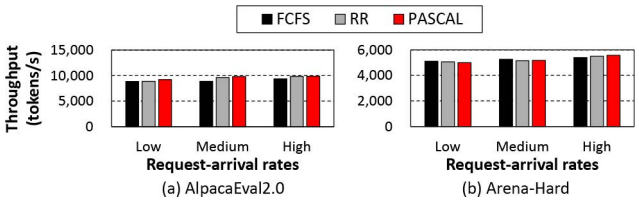

Fig. 12: Serving throughput across different request-arrival rates.

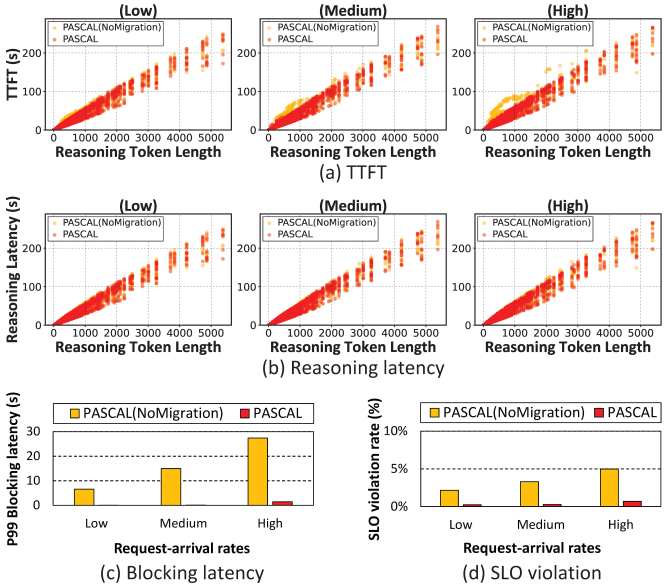

Fig. 13: Evaluation of (a) TTFT, (b) reasoning latency, (c) P99 blocking latency, and (d) SLO violation rate on the AlpacaEval2.0 dataset when PASCAL's inter-instance request migration feature is disabled (PASCAL(NoMigration)).

quest. PASCAL maintains a consistent phase-aware scheduling policy across all requests: reasoning phases always preempt answering phases, and requests migrate to the instance with the least reasoning requests at phase transitions. This avoids head-of-line blocking and keeps tail TTFT low across all datasets and traffic loads. Compared to FCFS, PASCAL cuts tail TTFT by up to 61% on AlpacaEval2.0 and 72% on Arena-Hard, with absolute tail TTFT reductions of 43.22 sec and 64.21 sec, respectively. Compared to RR, PASCAL reduces tail TTFT by up to 89.91 sec and 72.73 sec, a 33% and 29% reduction thereby significantly improving user experience.

It is worth noting that there exists a small number of cases where PASCAL causes minor TTFT degradation, but the absolute increases are small. For example, in AlpacaEval2.0 the worst case is a 13.30 sec increase vs. FCFS (6.12% higher TTFT) and an 8.25 sec increase vs. RR (9.23% higher TTFT). This slight degradation is due to increased preemption among answering requests competing for limited GPU memory. Outside these rare outliers, PASCAL consistently maintains lowest TTFT compared to all baseline schedulers.

SLO violations. As shown in Figure 11, PASCAL consistently achieves lower or comparable answering-phase SLO violation than both baselines across all request-arrival rates. This advantage stems from PASCAL's SLO-aware instance-level scheduling, which routes answering requests to instances with minimal reasoning load, while also prioritizing reasoning phase through its hierarchical queue. At request-arrival rates higher than those in Figure 11, all schedulers inevitably see higher SLO violations, as no serving system can sustain performance once demand exceeds its resource capacity. However, PASCAL confines most degradation to SLO violations while keeping TTFT low, whereas the baseline schedulers degrade in both TTFT and SLO metrics. This resilience arises from PASCAL's strict reasoning-phase prioritization via its hierarchical scheduler.

Serving throughput. Figure 12 shows the overall throughput of the LLM serving system across various request-arrival rates and datasets. Throughput is measured as the total time required to generate all output tokens, including both reasoning and answering tokens. As shown, PASCAL achieves throughput comparable to both baseline schedulers, differing by no more than 3%. This result demonstrates that software-level scheduling decisions that account for the distinct reasoning and answering phases of reasoning-based LLMs can significantly improve user experience (in terms of TTFT and TPOT) without compromising overall throughput.

C. KV Cache Transfer Overhead

In our multi-instance server environment, multiple instances may simultaneously migrate requests transitioning into the answering phase (along with their associated KV caches) to the same target instance, causing bandwidth contention and potentially increasing TTFT and overall latency. In our evaluation, we observed P99 KV cache transfer latencies of 0.14 sec for AlpacaEval2.0 and 0.25 sec for Arena-Hard under high request-arrival rates. Since TTFT in these scenarios ranges from a few seconds to several hundred seconds, the impact of KV cache transfer overhead on end-to-end latency is negligible. These results demonstrate that, although KV cache migration introduces some overhead, the performance gains from intelligently migrating phase-transitioning requests across instances far outweigh its costs.

D. Discussions

Importance of migrating requests. To demonstrate the importance of migrating a request at the reasoning-to-answering boundary, we construct a variant of PASCAL that *disables* migration. This version (aka PASCAL(NoMigration)) retains the hierarchical queue design but pins every request to the GPU instance chosen by Algorithm 1; once the reasoning

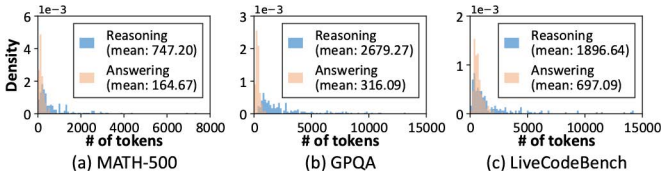

Fig. 14: Reasoning and answering token count distributions for (a) MATH-500 [17], (b) GPQA [41], and (c) LiveCodeBench [20]. Distributions are derived by querying each benchmark's prompts into the o4-mini model, following the methodology in Section V-A.

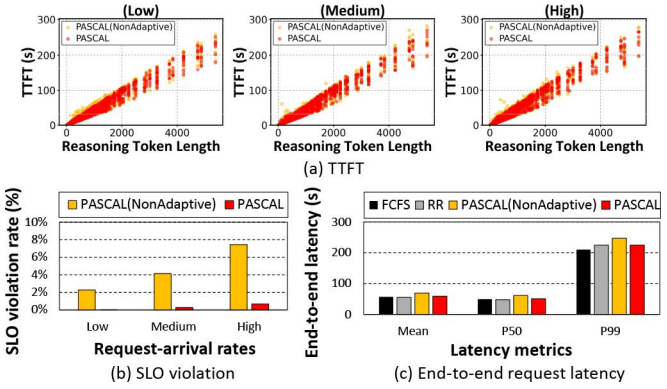

Fig. 15: Comparison of PASCAL(NonAdaptive) and PASCAL on the AlpacaEval2.0 dataset. (a) shows the TTFT distributions, (b) presents the SLO violation rates under varying request-arrival rates, and (c) compares the end-to-end latency (covering the entire execution from the reasoning phase to the answering phase) across multiple latency metrics under high request-arrival rates.

phase ends, no inter-instance migration occurs. Under PASCAL(NoMigration), the scheduler still prioritizes reasoning requests, but requests can still stall at the phase transition point. If GPU memory is fully utilized by high-priority reasoning requests, an answering request that is routed to the same instance's low-priority queue cannot get scheduled until the GPU memory is freed. Because PASCAL's original placement policy only considers the KV cache footprint during reasoning, it neglects the memory required for answering, thereby elevating the risk of SLO violations.

Figure 13 illustrates these effects. PASCAL(NoMigration) worsens tail TTFT under high request-arrival rates across a broad range of reasoning-token lengths (Figure 13(a)), even though reasoning phase latency remains almost unchanged (Figure 13(b)). To understand the cause, Figure 13(c) reports the latency between the moment a request transitions from reasoning to answering and the moment it is first scheduled. The P99 of this latency (referred to as *blocking latency*) reaches up to 27.39 sec in PASCAL(NoMigration), whereas PASCAL keeps it near zero. Overall, PASCAL(NoMigration) suffers from markedly higher answering phase SLO violation rates than PASCAL (Figure 13(d)), highlighting the value of PASCAL's SLO-aware request migration at phase boundaries.

Effectiveness of adaptive migration. To evaluate the effectiveness of PASCAL's adaptive migration policy, we compare our PASCAL against PASCAL(NonAdaptive), which turns

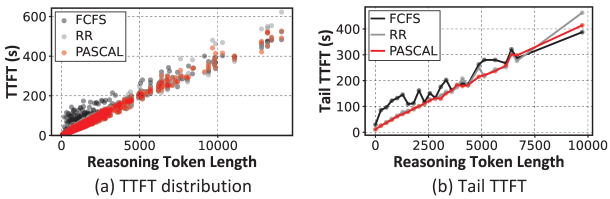

Fig. 16: (a) TTFT distribution and (b) tail TTFT by reasoning token length, with 50% of the Arena-Hard trace replaced by the reasoning-heavy datasets shown in Figure 14.

off adaptive migration and always triggers request migration at phase transitions, regardless of memory availability. Figure 15(a) and (b) show the TTFT distributions and SLO violation rates of PASCAL(NonAdaptive) versus PASCAL. Although the TTFT distributions appear similar, the SLO violation rate under PASCAL(NonAdaptive) rises sharply with increasing request arrival rates, reaching 7.45% compared to only 0.69% under PASCAL. This degradation occurs because PASCAL(NonAdaptive) ignores each instance's current memory state, migrating requests even when the target instance lacks sufficient GPU memory, while the current instance could have immediately served the answering phase without stalling (see Figure 7(a)). Although the instance-level scheduler excludes instances currently violating their SLOs, its decisions rely solely on the current token pacer state and cannot foresee future memory contention due to unpredictable output lengths. Consequently, such unnecessary migrations increase the risk of future SLO violations for answering requests on the target instance, a problem effectively mitigated by PASCAL's adaptive migration policy. Overall, compared to PASCAL, PASCAL(NonAdaptive) worsens median end-to-end latency by 20.1% and tail latency by 9.7% (Figure 15(c)).

Alternative reasoning datasets. Our evaluation so far has focused on chat applications where users expect detailed responses with long answering tokens. In contrast, problem-solving benchmarks [3], [17], [20], [21], [25], [41] feature lengthy reasoning phases and comparatively short answers. As shown in Figure 14(a)-(c), the number of reasoning tokens can reach as much as 8.48× higher than answering tokens in these datasets. In such *reasoning-heavy* datasets, PASCAL's benefits may diminish because answering phases are too short to cause significant scheduling contention, reducing the impact of phase-aware scheduling.

To this end, we also evaluate a scenario where 50% of Arena-Hard requests are replaced with reasoning-heavy requests sampled uniformly from MATH-500, GPQA, and LiveCodeBench. As shown in Figure 16, PASCAL reduces tail TTFT by up to 70% over FCFS for shorter reasoning segments by mitigating head-of-line blocking, where long reasoning monopolizes GPU memory until answering completes. Even for long-reasoning requests, TTFT rises only 6.8% compared to FCFS, a modest trade-off given the broader gains. Compared to RR, improvements are smaller due to dataset characteristics: short answering phases create minimal interference with reasoning even under baseline scheduling. RR's implicit per-request hierarchy (favoring reasoning over answering, as

described in Section V-B) further reduces contention. Thus, PASCAL's explicit phase isolation provides limited additional benefit in this setting. Nonetheless, PASCAL still reduces tail TTFT for certain token ranges by up to 13.9%, with worst-case degradation under 7.7%. SLO violation rates remain similar to RR and consistently lower than FCFS. Overall, PASCAL provides stable, competitive performance even when reasoning–answering contention is minimal.

VI. Related works

LLM serving optimization. Prior work [2], [23], [26], [27], [43], [44], [51] has optimized LLM serving for performance and SLO compliance. ORCA [51] introduces iteration-level scheduling with continuous batching to reduce queuing latency and improve throughput. vLLM [27] combines PagedAttention with continuous batching for high-throughput inference under fine-grained GPU memory management. Andes [31] proposes a QoE-aware preemptive scheduler for streaming services to preserve user-perceived responsiveness. Llumnix [44] supports priority-aware migration by assigning virtual memory usage to requests, guiding dynamic rescheduling to balance load and prioritize high-SLO requests. None of these systems address the unique characteristics of the reasoning phase in reasoning LLMs, leaving TTFT optimization largely unexplored.

Reasoning-based LLM inference. Recent work has tackled inefficiencies in long-chain reasoning. ThinkPrune [19] reduces redundant reasoning steps via reinforcement learning to shorten token sequences without sacrificing performance. Dynasor [13] complements this by predicting requests that are likely to require extended reasoning and allocating additional compute resources accordingly. For requests predicted to have reached a stable or confident reasoning state, Dynasor terminates decoding early to save computation. In contrast, PASCAL is a scheduling framework that exploits the asymmetric resource demands of reasoning versus answering phases, enabling priority-aware execution and dynamic migration across serving instances without modifying models or adding inference-time heuristics. Rather than manipulating the reasoning phase itself, PASCAL introduces a novel phase-aware scheduling mechanism that distinguishes between the two phases while keeping the execution pipeline intact, making it orthogonal to reasoning-token reduction methods [13], [19].

Disaggregated LLM inference. Several prior works [11], [39], [54] leverage the distinct compute characteristics of the prefill and decode stages by disaggregating these stages across dedicated hardware, improving energy efficiency and reducing request contention. DistServe [54], for example, partitions the inference pipeline based on the observation that prefill and decoding interfere with each other due to their differing resource demands, assigning each stage to hardware configurations that better satisfy their stage-specific SLO constraints. In contrast to these disaggregated LLM serving systems, which focus on conventional LLMs, our work targets emerging reasoning-based LLMs, where the decoding stage itself consists of two semantically distinct phases: reasoning and answering. The key contribution of PASCAL is identifying this phase-based execution behavior and intelligently scheduling decoding requests to minimize interference. This insight is orthogonal to existing disaggregated inference systems and can be layered on top of them for additional performance gains.

VII. Future work

PASCAL in heterogeneous system architectures. PASCAL focuses on GPU-centric LLM serving, consistent with many representative prior works [2], [27], [39], [54] where all computations are executed exclusively on GPUs and the CPU is used primarily as a backing storage for offloaded tensors, without participating in the LLM computation process. Meanwhile, several recent studies have explored a *heterogeneous* computing model for LLM inference, in which the CPU collaborates with the GPU to accelerate LLMs [24], [33]. More recently, the introduction of Rubin CPX [35]—a prefill-optimized GPU designed to complement the standard (now relatively decode-optimized) Rubin GPU [36] by handling the compute-bound prefill stage—has gained attention as it presents opportunity to balance compute and memory utilization across heterogeneous GPUs (i.e., prefill-optimized vs. decode-optimized).

Applying PASCAL's phase-aware scheduling algorithm on top of these emerging heterogeneous compute devices introduces unique design challenges. For instance, offloading KV caches, activations, or weights must be handled more carefully, as these data may still be needed for ongoing computations depending on how compute and memory operations are partitioned across devices, as well as how scheduling policies and inter-device coherence are orchestrated. A deeper exploration of PASCAL in such heterogeneous environments is beyond the scope of this work and is left for future investigation.

Explicit resource partitioning across reasoning and answering phases. In prior works such as DistServe [54] and Splitwise [39], the prefill (compute-bound) and decoding (memory-bound) stages exhibit fundamentally distinct computational characteristics, which makes explicit resource partitioning beneficial. Specifically, DistServe emphasizes the interference between the prefill and decoding stages that stems from their per-step latency difference, where the slower prefill stage delays decoding progress within the same iteration, while Splitwise leverages hardware heterogeneity to better suit each stage's computational characteristics. In contrast, under PASCAL, both the reasoning and answering phases belong to the *same* decoding stage. Thus, applying a DistServe-style explicit partitioning offers little benefit, as the two phases have similar per-step latencies and do not experience the iteration-level interference that motivates stage separation. Similarly, applying a Splitwise-style explicit resource partitioning is less effective, since both phases share comparable computational characteristics and are equally well-suited to the same hardware resources, yielding no efficiency gain from assigning them to different devices. Consequently, the potential benefits of explicitly partitioning resources across the reasoning and answering phases is questionable, while its drawbacks become more pronounced. A deeper exploration of this design space is left for future work.

VIII. Conclusion

In this paper, we introduce a phase-aware scheduling algorithm for reasoning-based LLM serving systems. Based on our key observation that reasoning latency dominates Time-To-First-Token (TTFT), PASCAL's scheduler applies lightweight phase detection and priority assignment during decoding, enabling our proposed system to honor the asymmetric performance sensitivities of reasoning versus answering tokens. A hierarchical intra-instance queue, combined with SLO-aware inter-instance migration, minimizes tail TTFT while preserving answer-phase token throughput. Consequently, PASCAL delivers consistently low tail latency without sacrificing overall throughput or answer quality.

Acknowledgment

This work was partly supported by Institute of Information & Communications Technology Planning & Evaluation(IITP) grant funded by the Korea government(MSIT) (No.RS-2024-00438851, (SW Starlab) High-performance Privacy-preserving Machine Learning System and System Software), (No.RS-2025-02264029, Implementation and Validation of an AI Semiconductor-Based Data Center Composable Cluster Infrastructure, 30%), (No.RS-2025-02214652, Development of SoC Technology for AI Semiconductor-Converged Pooled Storage/Memory), (No. RS-2024-00457882, AI Research Hub Project), Samsung Research Funding Center of Samsung Electronics (SRFC-IT2402-03), and IITP under the Graduate School of Artificial Intelligence Semiconductor(IITP-2026-RS-2023-00256472) grant funded by the Korea government(MSIT). We also thank Yunjae Lee for his constructive feedback that helped improve this paper. Minsoo Rhu is the corresponding author.

References

[1] A. Agrawal, N. Kedia, J. Mohan, A. Panwar, N. Kwatra, B. Gulavani, R. Ramjee, and A. Tumanov, "Vidur: A Large-Scale Simulation Framework For LLM Inference," in *The Eighth Annual Conference on Machine Learning and Systems (MLSys)*, 2024.

[2] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. Gulavani, A. Tumanov, and R. Ramjee, "Taming Throughput-latency Tradeoff in LLM Inference with Sarathi-Serve," in *Proceedings of the USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.

[3] V. AI, https://www.vals.ai/benchmarks/aime, 2025.

[4] K. Alizadeh, S. I. Mirzadeh, D. Belenko, S. Khatamifard, M. Cho, C. C. Del Mundo, M. Rastegari, and M. Farajtabar, "LLM in a flash: Efficient Large Language Model Inference with Limited Memory," in *Proceedings of the ACL (Association for Computational Linguistics)*, 2024.

[5] R. Anil, S. Borgeaud, J.-B. Alayrac, J. Yu, R. Soricut, J. Schalkwyk, A. M. Dai, A. Hauth, K. Millican, D. Silver, M. Johnson, I. Antonoglou, J. Schrittwieser, A. Glaese, J. Chen, E. Pitler, T. Lillicrap, A. Lazaridou, O. Firat, J. Molloy, M. Isard, P. R. Barham, T. Hennigan, B. Lee, F. Viola, M. Reynolds, Y. Xu, R. Doherty, E. Collins, C. Meyer, E. Rutherford, E. Moreira, K. Ayoub, M. Goel, J. Krawczyk, C. Du, E. Chi, H.-T. Cheng, E. Ni, P. Shah, P. Kane, B. Chan, M. Faruqui, A. Severyn, H. Lin, Y. Li, Y. Cheng, A. Ittycheriah, M. Mahdieh, M. Chen, P. Sun, D. Tran, S. Bagri, B. Lakshminarayanan, J. Liu, A. Orban, F. Güra, H. Zhou, X. Song, A. Boffy, H. Ganapathy, S. Zheng, H. Choe, Ágoston Weisz, T. Zhu, Y. Lu, S. Gopal, J. Kahn, M. Kula, J. Pitman, R. Shah, E. Taropa, M. A. Merey, M. Baeuml, Z. Chen, L. E. Shafey, Y. Zhang, O. Sercinoglu, G. Tucker, E. Piqueras, M. Krikun, I. Barr, N. Savinov, I. Danihelka, B. Roelofs, A. White, A. Andreassen, T. von Glehn, L. Yagati, M. Kazemi, L. Gonzalez, M. Khalman, J. Sygnowski, A. Frechette, C. Smith, L. Culp, L. Proleev, Y. Luan, X. Chen, J. Lottes, N. Schucher, F. Lebron, A. Rrustemi, N. Clay, P. Crone, T. Kocisky, J. Zhao, B. Perz, D. Yu, H. Howard, A. Bloniarz, J. W. Rae, H. Lu, L. Sifre, M. Maggioni, F. Alcober, D. Garrette, M. Barnes, S. Thakoor, J. Austin, G. Barth-Maron, W. Wong, R. Joshi, R. Chaabouni, D. Fatiha, A. Ahuja, G. S. Tomar, E. Senter, M. Chadwick, I. Kornakov, N. Attaluri, I. Iturrate, R. Liu, Y. Li, S. Cogan, J. Chen, C. Jia, C. Gu, Q. Zhang, J. Grimstad, A. J. Hartman, X. Garcia, T. S. Pillai, J. Devlin, M. Laskin, D. de Las Casas, D. Valter, C. Tao, L. Blanco, A. P. Badia, D. Reitter, M. Chen, J. Brennan, C. Rivera, S. Brin, S. Iqbal, G. Surita, J. Labanowski, A. Rao, S. Winkler, E. Parisotto, Y. Gu, K. Olszewska, R. Addanki, A. Miech, A. Louis, D. Teplyashin, G. Brown, E. Catt, J. Balaguer, J. Xiang, P. Wang, Z. Ashwood, A. Briukhov, A. Webson, S. Ganapathy, S. Sanghavi, A. Kannan, M.-W. Chang, A. Stjerngren, J. Djolonga, Y. Sun, A. Bapna, M. Aitchison, P. Pejman, H. Michalewski, T. Yu, C. Wang, J. Love, J. Ahn, D. Bloxwich, K. Han, P. Humphreys, T. Sellam, J. Bradbury, V. Godbole, S. Samangooei, B. Damoc, A. Kaskasoli, S. M. R. Arnold, V. Vasudevan, S. Agrawal, J. Riesa, D. Lepikhin, R. Tanburn, S. Srinivasan, H. Lim, S. Hodkinson, P. Shyam, J. Ferret, S. Hand, A. Garg, T. L. Paine, J. Li, Y. Li, M. Giang, A. Neitz, Z. Abbas, S. York, M. Reid, E. Cole, A. Chowdhery, D. Das, D. Rogozińska, V. Nikolaev, P. Sprechmann, Z. Nado, L. Zilka, F. Prost, L. He, M. Monteiro, G. Mishra, C. Welty, J. Newlan, D. Jia, M. Allamanis, C. H. Hu, R. de Liedekerke, J. Gilmer, C. Saroufim, S. Rijhwani, S. Hou, D. Shrivastava, A. Baddepudi, A. Goldin, A. Ozturel, A. Cassirer, Y. Xu, D. Sohn, D. Sachan, R. K. Amplayo, C. Swanson, D. Petrova, S. Narayan, A. Guez, S. Brahma, J. Landon, M. Patel, R. Zhao, K. Villela, L. Wang, W. Jia, M. Rahtz, M. Giménez, L. Yeung, J. Keeling, P. Georgiev, D. Mincu, B. Wu, S. Haykal, R. Saputro, K. Vodrahalli, J. Qin, Z. Cankara, A. Sharma, N. Fernando, W. Hawkins, B. Neyshabur, S. Kim, A. Hutter, P. Agrawal, A. Castro-Ros, G. van den Driessche, T. Wang, F. Yang, S. yiin Chang, P. Komarek, R. McIlroy, M. Lučić, G. Zhang, W. Farhan, M. Sharman, P. Natsev, P. Michel, Y. Bansal, S. Qiao, K. Cao, S. Shakeri, C. Butterfield, J. Chung, P. K. Rubenstein, S. Agrawal, A. Mensch, K. Soparkar, K. Lenc, T. Chung, A. Pope, L. Maggiore, J. Kay, P. Jhakra, S. Wang, J. Maynez, M. Phuong, T. Tobin, A. Tacchetti, M. Trebacz, K. Robinson, Y. Katariya, S. Riedel, P. Bailey, K. Xiao, N. Ghelani, L. Aroyo, A. Slone, N. Houlsby, X. Xiong, Z. Yang, E. Gribovskaya, J. Adler, M. Wirth, L. Lee, M. Li, T. Kagohara, J. Pavagadhi, S. Bridgers, A. Bortsova, S. Ghemawat, Z. Ahmed, T. Liu, R. Powell, V. Bolina, M. Iinuma, P. Zablotskaia, J. Besley, D.-W. Chung, T. Dozat, R. Comanescu, X. Si, J. Greer, G. Su, M. Polacek, R. L. Kaufman, S. Tokumine, H. Hu, E. Buchatskaya, Y. Miao, M. Elhawaty, A. Siddhant, N. Tomasev, J. Xing, C. Greer, H. Miller, S. Ashraf, A. Roy, Z. Zhang, A. Ma, A. Filos, M. Besta, R. Blevins, T. Klimenko, C.-K. Yeh, S. Changpinyo, J. Mu, O. Chang, M. Pajarskas, C. Muir, V. Cohen, C. L. Lan, K. Haridasan, A. Marathe, S. Hansen, S. Douglas, R. Samuel, M. Wang, S. Austin, C. Lan, J. Jiang, J. Chiu, J. A. Lorenzo, L. L. Sjösund, S. Cevey, Z. Gleicher, T. Avrahami, A. Boral, H. Srinivasan, V. Selo, R. May, K. Aisopos, L. Hussenot, L. B. Soares, K. Baumli, M. B. Chang, A. Recasens, B. Caine, A. Pritzel, F. Pavetic, F. Pardo, A. Gergely, J. Frye, V. Ramasesh, D. Horgan, K. Badola, N. Kassner, S. Roy, E. Dyer, V. C. Campos, A. Tomala, Y. Tang, D. E. Badawy, E. White, B. Mustafa, O. Lang, A. Jindal, S. Vikram, Z. Gong, S. Caelles, R. Hemsley, G. Thornton, F. Feng, W. Stokowiec, C. Zheng, P. Thacker, Çağlar Ünlü, Z. Zhang, M. Saleh, J. Svensson, M. Bileschi, P. Patil, A. Anand, R. Ring, K. Tsihlas, A. Vezer, M. Selvi, T. Shevlane, M. Rodriguez, T. Kwiatkowski, S. Daruki, K. Rong, A. Dafoe, N. FitzGerald, K. Gu-Lemberg, M. Khan, L. A. Hendricks, M. Pellat, V. Feinberg, J. Cobon-Kerr, T. Sainath, M. Rauh, S. H. Hashemi, R. Ives, Y. Hasson, E. Noland, Y. Cao, N. Byrd, L. Hou, Q. Wang, T. Sottiaux, M. Paganini, J.-B. Lespiau, A. Moufarek, S. Hassan, K. Shivakumar, J. van Amersfoort, A. Mandhane, P. Joshi, A. Goyal, M. Tung, A. Brock, H. Sheahan, V. Misra, C. Li, N. Rakićević, M. Dehghani, F. Liu, S. Mittal, J. Oh, S. Noury, E. Sezener, F. Huot, M. Lamm, N. D. Cao, C. Chen, S. Mudgal, R. Stella, K. Brooks, G. Vasudevan, C. Liu, M. Chain, N. Melinkeri, A. Cohen, V. Wang, K. Seymore, S. Zubkov, R. Goel, S. Yue, S. Krishnakumaran, B. Albert, N. Hurley, M. Sano, A. Mohananey, J. Joughin, E. Filonov, T. Kepa, Y. Eldawy, J. Lim, R. Rishi, S. Badiezadegan, T. Bos, J. Chang, S. Jain, S. G. S. Padmanabhan, S. Puttagunta, K. Krishna, L. Baker, N. Kalb, V. Bedapudi, A. Kurzrok, S. Lei, A. Yu, O. Litvin, X. Zhou, Z. Wu, S. Sobell, A. Siciliano, A. Papir, R. Neale, J. Bragagnolo,

T. Toor, T. Chen, V. Anklin, F. Wang, R. Feng, M. Gholami, K. Ling, L. Liu, J. Walter, H. Moghaddam, A. Kishore, J. Adamek, T. Mercado, J. Mallinson, S. Wandekar, S. Cagle, E. Ofek, G. Garrido, C. Lombriser, M. Mukha, B. Sun, H. R. Mohammad, J. Matak, Y. Qian, V. Peswani, P. Janus, Q. Yuan, L. Schelin, O. David, A. Garg, Y. He, O. Duzhyi, A. Älgmyr, T. Lottaz, Q. Li, V. Yadav, L. Xu, A. Chinien, R. Shivanna, A. Chuklin, J. Li, C. Spadine, T. Wolfe, K. Mohamed, S. Das, Z. Dai, K. He, D. von Dincklage, S. Upadhyay, A. Maurya, L. Chi, S. Krause, K. Salama, P. G. Rabinovitch, P. K. R. M, A. Selvan, M. Dektiarev, G. Ghiasi, E. Guven, H. Gupta, B. Liu, D. Sharma, I. H. Shtacher, S. Paul, O. Akerlund, F.-X. Aubet, T. Huang, C. Zhu, E. Zhu, E. Teixeira, M. Fritze, F. Bertolini, L.-E. Marinescu, M. Bölle, D. Paulus, K. Gupta, T. Latkar, M. Chang, J. Sanders, R. Wilson, X. Wu, Y.-X. Tan, L. N. Thiet, T. Doshi, S. Lall, S. Mishra, W. Chen, T. Luong, S. Benjamin, J. Lee, E. Andrejczuk, D. Rabiej, V. Ranjan, K. Styrc, P. Yin, J. Simon, M. R. Harriott, M. Bansal, A. Robsky, G. Bacon, D. Greene, D. Mirylenka, C. Zhou, O. Sarvana, A. Goyal, S. Andermatt, P. Siegler, B. Horn, A. Israel, F. Pongetti, C.-W. L. Chen, M. Selvatici, P. Silva, K. Wang, J. Tolins, K. Guu, R. Yogev, X. Cai, A. Agostini, M. Shah, H. Nguyen, N. Donnaile, S. Pereira, L. Friso, A. Stambler, A. Kurzrok, C. Kuang, Y. Romanikhin, M. Geller, Z. Yan, K. Jang, C.-C. Lee, W. Fica, E. Malmi, Q. Tan, D. Banica, D. Balle, R. Pham, Y. Huang, D. Avram, H. Shi, J. Singh, C. Hidey, N. Ahuja, P. Saxena, D. Dooley, S. P. Potharaju, E. O'Neill, A. Gokulchandran, R. Foley, K. Zhao, M. Dusenberry, Y. Liu, P. Mehta, R. Kotikalapudi, C. Safranek-Shrader, A. Goodman, J. Kessinger, E. Globen, P. Kolhar, C. Gorgolewski, A. Ibrahim, Y. Song, A. Eichenbaum, T. Brovelli, S. Potluri, P. Lahoti, C. Baetu, A. Ghorbani, C. Chen, A. Crawford, S. Pal, M. Sridhar, P. Gurita, A. Mujika, I. Petrovski, P.-L. Cedoz, C. Li, S. Chen, N. D. Santo, S. Goyal, J. Punjabi, K. Kappaganthu, C. Kwak, P. LV, S. Velury, H. Choudhury, J. Hall, P. Shah, R. Figueira, M. Thomas, M. Lu, T. Zhou, C. Kumar, T. Jurdi, S. Chikkerur, Y. Ma, A. Yu, S. Kwak, V. Ähdel, S. Rajayogam, T. Choma, F. Liu, A. Barua, C. Ji, J. H. Park, V. Hellendoorn, A. Bailey, T. Bilal, H. Zhou, M. Khatir, C. Sutton, W. Rzadkowski, F. Macintosh, R. Vij, K. Shagin, P. Medina, C. Liang, J. Zhou, P. Shah, Y. Bi, A. Dankovics, S. Banga, S. Lehmann, M. Bredesen, Z. Lin, J. E. Hoffmann, J. Lai, R. Chung, K. Yang, N. Balani, A. Bražinskas, A. Sozanschi, M. Hayes, H. F. Alcalde, P. Makarov, W. Chen, A. Stella, L. Snijders, M. Mandl, A. Kärrman, P. Nowak, X. Wu, A. Dyck, K. Vaidyanathan, R. R, J. Mallet, M. Rudominer, E. Johnston, S. Mittal, A. Udathu, J. Christensen, V. Verma, Z. Irving, A. Santucci, G. Elsayed, E. Davoodi, M. Georgiev, I. Tenney, N. Hua, G. Cideron, E. Leurent, M. Alnahlawi, I. Georgescu, N. Wei, I. Zheng, D. Scandinaro, H. Jiang, J. Snoek, M. Sundararajan, X. Wang, Z. Ontiveros, I. Karo, J. Cole, V. Rajashekhar, L. Tumeh, E. Ben-David, R. Jain, J. Uesato, R. Datta, O. Bunyan, S. Wu, J. Zhang, P. Stanczyk, Y. Zhang, D. Steiner, S. Naskar, M. Azzam, M. Johnson, A. Paszke, C.-C. Chiu, J. S. Elias, A. Mohiuddin, F. Muhammad, J. Miao, A. Lee, N. Vieillard, J. Park, J. Zhang, J. Stanway, D. Garmon, A. Karmarkar, Z. Dong, J. Lee, A. Kumar, L. Zhou, J. Evens, W. Isaac, G. Irving, E. Loper, M. Fink, I. Arkatkar, N. Chen, I. Shafran, I. Petrychenko, Z. Chen, J. Jia, A. Levskaya, Z. Zhu, P. Grabowski, Y. Mao, A. Magni, K. Yao, J. Snaider, N. Casagrande, E. Palmer, P. Suganthan, A. Castaño, I. Giannoumis, W. Kim, M. Rybiński, A. Sreevatsa, J. Prendki, D. Soergel, A. Goedeckemeyer, W. Gierke, M. Jafari, M. Gaba, J. Wiesner, D. G. Wright, Y. Wei, H. Vashisht, Y. Kulizhskaya, J. Hoover, M. Le, L. Li, C. Iwuanyanwu, L. Liu, K. Ramirez, A. Khorlin, A. Cui, T. LIN, M. Wu, R. Aguilar, K. Pallo, A. Chakladar, G. Perng, E. A. Abellan, M. Zhang, I. Dasgupta, N. Kushman, I. Penchev, A. Repina, X. Wu, T. van der Weide, P. Ponnapalli, C. Kaplan, J. Simsa, S. Li, O. Dousse, F. Yang, J. Piper, N. Ie, R. Pasumarthi, N. Lintz, A. Vijayakumar, D. Andor, P. Valenzuela, M. Lui, C. Paduraru, D. Peng, K. Lee, S. Zhang, S. Greene, D. D. Nguyen, P. Kurylowicz, C. Hardin, L. Dixon, L. Janzer, K. Choo, Z. Feng, B. Zhang, A. Singhal, D. Du, D. McKinnon, N. Antropova, T. Bolukbasi, O. Keller, D. Reid, D. Finchelstein, M. A. Raad, R. Crocker, P. Hawkins, R. Dadashi, C. Gaffney, K. Franko, A. Bulanova, R. Leblond, S. Chung, H. Askham, L. C. Cobo, K. Xu, F. Fischer, J. Xu, C. Sorokin, C. Alberti, C.-C. Lin, C. Evans, A. Dimitriev, H. Forbes, D. Banarse, Z. Tung, M. Omernick, C. Bishop, R. Sterneck, R. Jain, J. Xia, E. Amid, F. Piccinno, X. Wang, P. Banzal, D. J. Mankowitz, A. Polozov, V. Krakovna, S. Brown, M. Bateni, D. Duan, V. Firoiu, M. Thotakuri, T. Natan, M. Geist, S. tan Girgin, H. Li, J. Ye, O. Roval, R. Tojo, M. Kwong, J. Lee-Thorp, C. Yew, D. Sinopalnikov, S. Ramos, J. Mellor, A. Sharma, K. Wu, D. Miller, N. Sonnerat, D. Vnukov, R. Greig, J. Beattie, E. Caveness, L. Bai, J. Eisenschlos, A. Korchemniy, T. Tsai, M. Jasarevic, W. Kong, P. Dao, Z. Zheng, F. Liu, F. Yang, R. Zhu, T. H. Teh, J. Sanmiya, E. Gladchenko, N. Trdin, D. Toyama, E. Rosen, S. Tavakkol, L. Xue, C. Elkind, O. Woodman, J. Carpenter, G. Papamakarios, R. Kemp, S. Kafle, T. Grunina, R. Sinha, A. Talbert, D. Wu, D. Owusu-Afriyie, C. Du, C. Thornton, J. Pont-Tuset, P. Narayana, J. Li, S. Fatehi, J. Wieting, O. Ajmeri, B. Uria, Y. Ko, L. Knight, A. Héliou, N. Niu, S. Gu, C. Pang, Y. Li, N. Levine, A. Stolovich, R. Santamaria-Fernandez, S. Goenka, W. Yustalim, R. Strudel, A. Elqursh, C. Deck, H. Lee, Z. Li, K. Levin, R. Hoffmann, D. Holtmann-Rice, O. Bachem, S. Arora, C. Koh, S. H. Yeganeh, S. Põder, M. Tariq, Y. Sun, L. Ionita, M. Seyedhosseini, P. Tafti, Z. Liu, A. Gulati, J. Liu, X. Ye, B. Chrzaszcz, L. Wang, N. Sethi, T. Li, B. Brown, S. Singh, W. Fan, A. Parisi, J. Stanton, V. Koverkathu, C. A. Choquette-Choo, Y. Li, T. Lu, A. Ittycheriah, P. Shroff, M. Varadarajan, S. Bahargam, R. Willoughby, D. Gaddy, G. Desjardins, M. Cornero, B. Robenek, B. Mittal, B. Albrecht, A. Shenoy, F. Moiseev, H. Jacobsson, A. Ghaffarkhah, M. Rivière, A. Walton, C. Crepy, A. Parrish, Z. Zhou, C. Farabet, C. Radebaugh, P. Srinivasan, C. van der Salm, A. Fidjeland, S. Scellato, E. Latorre-Chimoto, H. Klimczak-Plucińska, D. Bridson, D. de Cesare, T. Hudson, P. Mendolicchio, L. Walker, A. Morris, M. Mauger, A. Guseynov, A. Reid, S. Odoom, L. Loher, V. Cotruta, M. Yenugula, D. Grewe, A. Petrushkina, T. Duerig, A. Sanchez, S. Yadlowsky, A. Shen, A. Globerson, L. Webb, S. Dua, D. Li, S. Bhupatiraju, D. Hurt, H. Qureshi, A. Agarwal, T. Shani, M. Eyal, A. Khare, S. R. Belle, L. Wang, C. Tekur, M. S. Kale, J. Wei, R. Sang, B. Saeta, T. Liechty, Y. Sun, Y. Zhao, S. Lee, P. Nayak, D. Fritz, M. R. Vuyyuru, J. Aslanides, N. Vyas, M. Wicke, X. Ma, E. Eltyshev, N. Martin, H. Cate, J. Manyika, K. Amiri, Y. Kim, X. Xiong, K. Kang, F. Luisier, N. Tripuraneni, D. Madras, M. Guo, A. Waters, O. Wang, J. Ainslie, J. Baldridge, H. Zhang, G. Pruthi, J. Bauer, F. Yang, R. Mansour, J. Gelman, Y. Xu, G. Polovets, J. Liu, H. Cai, W. Chen, X. Sheng, E. Xue, S. Ozair, C. Angermueller, X. Li, A. Sinha, W. Wang, J. Wiesinger, E. Koukoumidis, Y. Tian, A. Iyer, M. Gurumurthy, M. Goldenson, P. Shah, M. Blake, H. Yu, A. Urbanowicz, J. Palomaki, C. Fernando, K. Durden, H. Mehta, N. Momchev, E. Rahimtoroghi, M. Georgaki, A. Raul, S. Ruder, M. Redshaw, J. Lee, D. Zhou, K. Jalan, D. Li, B. Hechtman, P. Schuh, M. Nasr, K. Milan, V. Mikulik, J. Franco, T. Green, N. Nguyen, J. Kelley, A. Mahendru, A. Hu, J. Howland, B. Vargas, J. Hui, K. Bansal, V. Rao, R. Ghiya, E. Wang, K. Ye, J. M. Sarr, M. M. Preston, M. Elish, S. Li, A. Kaku, J. Gupta, I. Pasupat, D.-C. Juan, M. Someswar, T. M., X. Chen, A. Amini, A. Fabrikant, E. Chu, X. Dong, A. Muthal, S. Buthpitiya, S. Jauhari, N. Hua, U. Khandelwal, A. Hitron, J. Ren, L. Rinaldi, S. Drath, A. Dabush, N.-J. Jiang, H. Godhia, U. Sachs, A. Chen, Y. Fan, H. Taitelbaum, H. Noga, Z. Dai, J. Wang, C. Liang, J. Hamer, C.-S. Ferng, C. Elkind, A. Atias, P. Lee, V. Listík, M. Carlen, J. van de Kerkhof, M. Pikus, K. Zaher, P. Müller, S. Zykova, R. Stefanec, V. Gatsko, C. Hirnschall, A. Sethi, X. F. Xu, C. Ahuja, B. Tsai, A. Stefanoiu, B. Feng, K. Dhandhania, M. Katyal, A. Gupta, A. Parulekar, D. Pitta, J. Zhao, V. Bhatia, Y. Bhavnani, O. Alhadlaq, X. Li, P. Danenberg, D. Tu, A. Pine, V. Filippova, A. Ghosh, B. Limonchik, B. Urala, C. K. Lanka, D. Clive, Y. Sun, E. Li, H. Wu, K. Hongtongsak, I. Li, K. Thakkar, K. Omarov, K. Majmundar, M. Alverson, M. Kucharski, M. Patel, M. Jain, M. Zabelin, P. Pelagatti, R. Kohli, S. Kumar, J. Kim, S. Sankar, V. Shah, L. Ramachandruni, X. Zeng, B. Bariach, L. Weidinger, T. Vu, A. Andreev, A. He, K. Hui, S. Kashem, A. Subramanya, S. Hsiao, D. Hassabis, K. Kavukcuoglu, A. Sadovsky, Q. Le, T. Strohman, Y. Wu, S. Petrov, J. Dean, and O. Vinyals, "Gemini: A Family of Highly Capable Multimodal Models," in *arxiv.org*, 2023.

[6] J. Bang, Y. Choi, M. Kim, Y. Kim, and M. Rhu, "vTrain: A Simulation Framework for Evaluating Cost-effective and Compute-optimal Large Language Model Training," in *Proceedings of the International Symposium on Microarchitecture (MICRO)*, 2024.

[7] Z. Cai, Y. Zhang, B. Gao, Y. Liu, Y. Li, T. Liu, K. Lu, W. Xiong, Y. Dong, J. Hu *et al.*, "PyramidKV: Dynamic KV Cache Compression based on Pyramidal Information Funneling," in *arxiv.org*, 2024.

[8] A. Chowdhery, S. Narang, J. Devlin, M. Bosma, G. Mishra, A. Roberts, P. Barham, H. W. Chung, C. Sutton, S. Gehrmann, P. Schuh, K. Shi, S. Tsvyashchenko, J. Maynez, A. Rao, P. Barnes, Y. Tay, N. Shazeer, V. Prabhakaran, E. Reif, N. Du, B. Hutchinson, R. Pope, J. Bradbury, J. Austin, M. Isard, G. Gur-Ari, P. Yin, T. Duke, A. Levskaya, S. Ghemawat, S. Dev, H. Michalewski, X. Garcia, V. Misra, K. Robinson, L. Fedus, D. Zhou, D. Ippolito, D. Luan, H. Lim, B. Zoph, A. Spiridonov,

R. Sepassi, D. Dohan, S. Agrawal, M. Omernick, A. M. Dai, T. S. Pillai, M. Pellat, A. Lewkowycz, E. Moreira, R. Child, O. Polozov, K. Lee, Z. Zhou, X. Wang, B. Saeta, M. Diaz, O. Firat, M. Catasta, J. Wei, K. Meier-Hellstern, D. Eck, J. Dean, S. Petrov, and N. Fiedel, "PaLM: Scaling Language Modeling with Pathways," in *arxiv.org*, 2022.

[9] DeepSeek-AI, "DeepSeek-R1-Distill-Qwen-32B," 2025. [Online]. Available: https://huggingface.co/deepseek-ai/DeepSeek-R1-Distill-Qwen-32B

[10] Y. Dubois, B. Galambosi, P. Liang, and T. B. Hashimoto, "Length-Controlled AlpacaEval: A Simple Way to Debias Automatic Evaluators," in *Conference on Language Modeling (COLM)*, 2024.

[11] J. Feng, Y. Huang, R. Zhang, S. Liang, M. Yan, and J. Wu, "WindServe: Efficient Phase-Disaggregated LLM Serving with Stream-based Dynamic Scheduling," in *Proceedings of the International Symposium on Computer Architecture (ISCA)*, 2025.

[12] Y. Fu, H. Peng, R. Schwartz, N. A. Smith, and D. Khashabi, "Automatic Chain of Thought Prompting in Large Language Models," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2023.

[13] Y. Fu, J. Chen, S. Zhu, Z. Fu, Z. Dai, A. Qiao, and H. Zhang, "Efficiently Serving LLM Reasoning Programs with Certaindex," in *arxiv.org*, 2024.

[14] R. Gong, S. Bai, S. Wu, Y. Fan, Z. Wang, X. Li, H. Yang, and X. Liu, "Past-Future Scheduler for LLM Serving under SLA Guarantees," in *Proceedings of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2025.

[15] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan, A. Yang, A. Fan, A. Goyal, A. Hartshorn, A. Yang, A. Mitra, A. Sravankumar, A. Korenev, A. Hinsvark, A. Rao, A. Zhang, A. Rodriguez, A. Gregerson, A. Spataru, B. Roziere, B. Biron, B. Tang, B. Chern, C. Caucheteux, C. Nayak, C. Bi, C. Marra, C. McConnell, C. Keller, C. Touret, C. Wu, C. Wong, C. C. Ferrer, C. Nikolaidis, D. Allonsius, D. Song, D. Pintz, D. Livshits, D. Wyatt, D. Esiobu, D. Choudhary, D. Mahajan, D. Garcia-Olano, D. Perino, D. Hupkes, E. Lakomkin, E. AlBadawy, E. Lobanova, E. Dinan, E. M. Smith, F. Radenovic, F. Guzmán, F. Zhang, G. Synnaeve, G. Lee, G. L. Anderson, G. Thattai, G. Nail, G. Mialon, G. Pang, G. Cucurell, H. Nguyen, H. Korevaar, H. Xu, H. Touvron, I. Zarov, I. A. Ibarra, I. Kloumann, I. Misra, I. Evtimov, J. Zhang, J. Copet, J. Lee, J. Geffert, J. Vranes, J. Park, J. Mahadeokar, J. Shah, J. van der Linde, J. Billock, J. Hong, J. Lee, J. Fu, J. Chi, J. Huang, J. Liu, J. Wang, J. Yu, J. Bitton, J. Spisak, J. Park, J. Rocca, J. Johnstun, J. Saxe, J. Jia, K. V. Alwala, K. Prasad, K. Upasani, K. Plawiak, K. Li, K. Heafield, K. Stone, K. El-Arini, K. Iyer, K. Malik, K. Chiu, K. Bhalla, K. Lakhotia, L. Rantala-Yeary, L. van der Maaten, L. Chen, L. Tan, L. Jenkins, L. Martin, L. Madaan, L. Malo, L. Blecher, L. Landzaat, L. de Oliveira, M. Muzzi, M. Pasupuleti, M. Singh, M. Paluri, M. Kardas, M. Tsimpoukelli, M. Oldham, M. Rita, M. Pavlova, M. Kambadur, M. Lewis, M. Si, M. K. Singh, M. Hassan, N. Goyal, N. Torabi, N. Bashlykov, N. Bogoychev, N. Chatterji, N. Zhang, O. Duchenne, O. Çelebi, P. Alrassy, P. Zhang, P. Li, P. Vasic, P. Weng, P. Bhargava, P. Dubal, P. Krishnan, P. S. Koura, P. Xu, Q. He, Q. Dong, R. Srinivasan, R. Ganapathy, R. Calderer, R. S. Cabral, R. Stojnic, R. Raileanu, R. Maheswari, R. Girdhar, R. Patel, R. Sauvestre, R. Polidoro, R. Sumbaly, R. Taylor, R. Silva, R. Hou, R. Wang, S. Hosseini, S. Chennabasappa, S. Singh, S. Bell, S. S. Kim, S. Edunov, S. Nie, S. Narang, S. Raparthy, S. Shen, S. Wan, S. Bhosale, S. Zhang, S. Vandenhende, S. Batra, S. Whitman, S. Sootla, S. Collot, S. Gururangan, S. Borodinsky, T. Herman, T. Fowler, T. Sheasha, T. Georgiou, T. Scialom, T. Speckbacher, T. Mihaylov, T. Xiao, U. Karn, V. Goswami, V. Gupta, V. Ramanathan, V. Kerkez, V. Gonguet, V. Do, V. Vogeti, V. Albiero, V. Petrovic, W. Chu, W. Xiong, W. Fu, W. Meers, X. Martinet, X. Wang, X. Wang, X. E. Tan, X. Xia, X. Xie, X. Jia, X. Wang, Y. Goldschlag, Y. Gaur, Y. Babaei, Y. Wen, Y. Song, Y. Zhang, Y. Li, Y. Mao, Z. D. Coudert, Z. Yan, Z. Chen, Z. Papakipos, A. Singh, A. Srivastava, A. Jain, A. Kelsey, A. Shajnfeld, A. Gangidi, A. Victoria, A. Goldstand, A. Menon, A. Sharma, A. Boesenberg, A. Baevski, A. Feinstein, A. Kallet, A. Sangani, A. Teo, A. Yunus, A. Lupu, A. Alvarado, A. Caples, A. Gu, A. Ho, A. Poulton, A. Ryan, A. Ramchandani, A. Dong, A. Franco, A. Goyal, A. Saraf, A. Chowdhury, A. Gabriel, A. Bharambe, A. Eisenman, A. Yazdan, B. James, B. Maurer, B. Leonhardi, B. Huang, B. Loyd, B. D. Paola, B. Paranjape, B. Liu, B. Wu, B. Ni, B. Hancock, B. Wasti, B. Spence, B. Stojkovic, B. Gamido, B. Montalvo, C. Parker, C. Burton, C. Mejia, C. Liu, C. Wang, C. Kim, C. Zhou, C. Hu, C.-H. Chu, C. Cai, C. Tindal, C. Feichtenhofer, C. Gao, D. Civin, D. Beaty, D. Kreymer, D. Li, D. Adkins, D. Xu, D. Testuggine, D. David, D. Parikh, D. Liskovich, D. Foss, D. Wang, D. Le, D. Holland, E. Dowling, E. Jamil, E. Montgomery, E. Presani, E. Hahn, E. Wood, E.-T. Le, E. Brinkman, E. Arcaute, E. Dunbar, E. Smothers, F. Sun, F. Kreuk, F. Tian, F. Kokkinos, F. Ozgenel, F. Caggioni, F. Kanayet, F. Seide, G. M. Florez, G. Schwarz, G. Badeer, G. Swee, G. Halpern, G. Herman, G. Sizov, Guangyi, Zhang, G. Lakshminarayanan, H. Inan, H. Shojanazeri, H. Zou, H. Wang, H. Zha, H. Habeeb, H. Rudolph, H. Suk, H. Aspegren, H. Goldman, H. Zhan, I. Damlaj, I. Molybog, I. Tufanov, I. Leontiadis, I.-E. Veliche, I. Gat, J. Weissman, J. Geboski, J. Kohli, J. Lam, J. Asher, J.-B. Gaya, J. Marcus, J. Tang, J. Chan, J. Zhen, J. Reizenstein, J. Teboul, J. Zhong, J. Jin, J. Yang, J. Cummings, J. Carvill, J. Shepard, J. McPhie, J. Torres, J. Ginsburg, J. Wang, K. Wu, K. H. U, K. Saxena, K. Khandelwal, K. Zand, K. Matosich, K. Veeraraghavan, K. Michelena, K. Li, K. Jagadeesh, K. Huang, K. Chawla, K. Huang, L. Chen, L. Garg, L. A, L. Silva, L. Bell, L. Zhang, L. Guo, L. Yu, L. Moshkovich, L. Wehrstedt, M. Khabsa, M. Avalani, M. Bhatt, M. Mankus, M. Hasson, M. Lennie, M. Reso, M. Groshev, M. Naumov, M. Lathi, M. Keneally, M. Liu, M. L. Seltzer, M. Valko, M. Restrepo, M. Patel, M. Vyatskov, M. Samvelyan, M. Clark, M. Macey, M. Wang, M. J. Hermoso, M. Metanat, M. Rastegari, M. Bansal, N. Santhanam, N. Parks, N. White, N. Bawa, N. Singhal, N. Egebo, N. Usunier, N. Mehta, N. P. Laptev, N. Dong, N. Cheng, O. Chernoguz, O. Hart, O. Salpekar, O. Kalinli, P. Kent, P. Parekh, P. Saab, P. Balaji, P. Rittner, P. Bontrager, P. Roux, P. Dollar, P. Zvyagina, P. Ratanchandani, P. Yuvraj, Q. Liang, R. Alao, R. Rodriguez, R. Ayub, R. Murthy, R. Nayani, R. Mitra, R. Parthasarathy, R. Li, R. Hogan, R. Battey, R. Wang, R. Howes, R. Rinott, S. Mehta, S. Siby, S. J. Bondu, S. Datta, S. Chugh, S. Hunt, S. Dhillon, S. Sidorov, S. Pan, S. Mahajan, S. Verma, S. Yamamoto, S. Ramaswamy, S. Lindsay, S. Lindsay, S. Feng, S. Lin, S. C. Zha, S. Patil, S. Shankar, S. Zhang, S. Zhang, S. Wang, S. Agarwal, S. Sajuyigbe, S. Chintala, S. Max, S. Chen, S. Kehoe, S. Satterfield, S. Govindaprasad, S. Gupta, S. Deng, S. Cho, S. Virk, S. Subramanian, S. Choudhury, S. Goldman, T. Remez, T. Glaser, T. Best, T. Koehler, T. Robinson, T. Li, T. Zhang, T. Matthews, T. Chou, T. Shaked, V. Vontimitta, V. Ajayi, V. Montanez, V. Mohan, V. S. Kumar, V. Mangla, V. Ionescu, V. Poenaru, V. T. Mihailescu, V. Ivanov, W. Li, W. Wang, W. Jiang, W. Bouaziz, W. Constable, X. Tang, X. Wu, X. Wang, X. Wu, X. Gao, Y. Kleinman, Y. Chen, Y. Hu, Y. Jia, Y. Qi, Y. Li, Y. Zhang, Y. Zhang, Y. Adi, Y. Nam, Yu, Wang, Y. Zhao, Y. Hao, Y. Qian, Y. Li, Y. He, Z. Rait, Z. DeVito, Z. Rosnbrick, Z. Wen, Z. Yang, Z. Zhao, and Z. Ma, "The Llama 3 Herd of Models," in *arxiv.org*, 2024.

[16] D. Guo, D. Yang, H. Zhang, J. Song, R. Zhang, R. Xu, Q. Zhu, S. Ma, P. Wang, X. Bi, X. Zhang, X. Yu, Y. Wu, Z. F. Wu, Z. Gou, Z. Shao, Z. Li, Z. Gao, A. Liu, B. Xue, B. Wang, B. Wu, B. Feng, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan, D. Dai, D. Chen, D. Ji, E. Li, F. Lin, F. Dai, F. Luo, G. Hao, G. Chen, G. Li, H. Zhang, H. Bao, H. Xu, H. Wang, H. Ding, H. Xin, H. Gao, H. Qu, H. Li, J. Guo, J. Li, J. Wang, J. Chen, J. Yuan, J. Qiu, J. Li, J. L. Cai, J. Ni, J. Liang, J. Chen, K. Dong, K. Hu, K. Gao, K. Guan, K. Huang, K. Yu, L. Wang, L. Zhang, L. Zhao, L. Wang, L. Zhang, L. Xu, L. Xia, M. Zhang, M. Zhang, M. Tang, M. Li, M. Wang, M. Li, N. Tian, P. Huang, P. Zhang, Q. Wang, Q. Chen, Q. Du, R. Ge, R. Zhang, R. Pan, R. Wang, R. J. Chen, R. L. Jin, R. Chen, S. Lu, S. Zhou, S. Chen, S. Ye, S. Wang, S. Yu, S. Zhou, S. Pan, S. S. Li, S. Zhou, S. Wu, S. Ye, T. Yun, T. Pei, T. Sun, T. Wang, W. Zeng, W. Zhao, W. Liu, W. Liang, W. Gao, W. Yu, W. Zhang, W. L. Xiao, W. An, X. Liu, X. Wang, X. Chen, X. Nie, X. Cheng, X. Liu, X. Xie, X. Liu, X. Yang, X. Li, X. Su, X. Lin, X. Q. Li, X. Jin, X. Shen, X. Chen, X. Sun, X. Wang, X. Song, X. Zhou, X. Wang, X. Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. Zhang, Y. Xu, Y. Li, Y. Zhao, Y. Sun, Y. Wang, Y. Yu, Y. Zhang, Y. Shi, Y. Xiong, Y. He, Y. Piao, Y. Wang, Y. Tan, Y. Ma, Y. Liu, Y. Guo, Y. Ou, Y. Wang, Y. Gong, Y. Zou, Y. He, Y. Xiong, Y. Luo, Y. You, Y. Liu, Y. Zhou, Y. X. Zhu, Y. Xu, Y. Huang, Y. Li, Y. Zheng, Y. Zhu, Y. Ma, Y. Tang, Y. Zha, Y. Yan, Z. Z. Ren, Z. Ren, Z. Sha, Z. Fu, Z. Xu, Z. Xie, Z. Zhang, Z. Hao, Z. Ma, Z. Yan, Z. Wu, Z. Gu, Z. Zhu, Z. Liu, Z. Li, Z. Xie, Z. Song, Z. Pan, Z. Huang, Z. Xu, Z. Zhang, and Z. Zhang, "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning," in *arxiv.org*, 2025.

[17] D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt, "Measuring Mathematical Problem Solving With the MATH Dataset," in *Proceedings of the International Conference on Neural Information Processing Systems (NeurIPS)*, 2021.

[18] C. Holmes, M. Tanaka, M. Wyatt, A. A. Awan, J. Rasley, S. Rajbhandari, R. Y. Aminabadi, H. Qin, A. Bakhtiari, L. Kurilenko, and Y. He,

"DeepSpeed-FastGen: High-throughput Text Generation for LLMs via MII and DeepSpeed-Inference," in *arxiv.org*, 2024.
[19] B. Hou, Y. Zhang, J. Ji, Y. Liu, K. Qian, J. Andreas, and S. Chang, "ThinkPrune: Pruning Long Chain-of-Thought of LLMs via Reinforcement Learning," in *arxiv.org*, 2025.
[20] N. Jain, K. Han, A. Gu, W.-D. Li, F. Yan, T. Zhang, S. Wang, A. Solar-Lezama, K. Sen, and I. Stoica, "LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code," in *arxiv.org*, 2024.
[21] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. R. Narasimhan, "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?" in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024.
[22] A. K. Kakolyris, D. Masouros, P. Vavaroutsos, S. Xydis, and D. Soudris, "throttLL'eM: Predictive GPU Throttling for Energy Efficient LLM Inference Serving ," in *Proceedings of the International Symposium on High-Performance Computer Architecture (HPCA)*, 2025.
[23] A. K. Kamath, R. Prabhu, J. Mohan, S. Peter, R. Ramjee, and A. Panwar, "POD-Attention: Unlocking Full Prefill-Decode Overlap for Faster LLM Inference," in *Proceedings of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2025.
[24] H. Kim, G. Ye, N. Wang, A. Yazdanbakhsh, and N. S. Kim, "Exploiting Intel Advanced Matrix Extensions (AMX) for Large Language Model Inference," in *IEEE Computer Architecture Letters*, 2024.
[25] S. Krishna, K. Krishna, A. Mohananey, S. Schwarcz, A. Stambler, S. Upadhyay, and M. Faruqui, "Fact, Fetch, and Reason: A Unified Evaluation of Retrieval-Augmented Generation," in *arxiv.org*, 2025.
[26] A. V. Kumar, G. Antichi, and R. Singh, "AQUA: Network-Accelerated Memory Offloading for LLMs in Scale-Up GPU Domains," in *Proceedings of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2025.
[27] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient Memory Management for Large Language Model Serving with PagedAttention," in *Proceedings of the ACM Symposium on Operating System Principles (SOSP)*, 2023.
[28] A. Lewkowycz, T. Khot, B. Dalvi, P. Shaw, D. Das, K. Lee, J. Chen, X. Wang, S. Das, E. Zelikman, B. Murdoch, E. Kokiopoulou, G. Ghosh, E. Chi, Q. V. Le, D. Zhou, W. Cohen, J. Gupta, and S. Oh, "MetaMath: A Benchmark for Evaluating Reasoning with Language Models," in *arxiv.org*, 2023.
[29] T. Li, W.-L. Chiang, E. Frick, L. Dunlap, T. Wu, B. Zhu, J. E. Gonzalez, and I. Stoica, "From Crowdsourced Data to High-Quality Benchmarks: Arena-Hard and BenchBuilder Pipeline," in *arxiv.org*, 2024.
[30] Y.-C. Lin, W. Kwon, R. Pineda, and F. N. Paravecino, "Toward High-performance LLM Serving: A Simulation-Based Approach for Identifying Optimal Parallelism," in *arxiv.org*, 2024.
[31] J. Liu, J.-W. Chung, Z. Wu, F. Lai, M. Lee, and M. Chowdhury, "Andes: Defining and Enhancing Quality-of-Experience in LLM-based Text Streaming Services," in *arxiv.org*, 2024.
[32] MLCommons, https://mlcommons.org/2025/09/small-llm-inference-5-1/, 2025.
[33] S. Na, G. Jeong, B. H. Ahn, J. Young, T. Krishna, and H. Kim, "Understanding Performance Implications of LLM Inference on CPUs," in *Proceedings of the International Symposium on Workload Characterization (IISWC)*, 2024.
[34] NVIDIA, "NVIDIA Dynamo Platform," https://developer.nvidia.com/dynamo, 2025.
[35] NVIDIA, "Rubin CPX," https://nvidianews.nvidia.com/news/nvidia-unveils-rubin-cpx-a-new-class-of-gpu-designed-for-massive-context-inference, 2025.
[36] NVIDIA, "Rubin platform," https://blogs.nvidia.com/blog/computex-2024-jensen-huang/, 2025.
[37] OpenAI, "GPT-4 Technical Report," in *arxiv.org*, 2023.
[38] OpenAI, https://cdn.openai.com/pdf/2221c875-02dc-4789-800b-e7758f3722c1/o3-and-o4-mini-system-card.pdf, 2025.
[39] P. Patel, E. Choukse, C. Zhang, A. Shah, Í. Goiri, S. Maleki, and R. Bianchini, "Splitwise: Efficient Generative LLM Inference Using Phase Splitting," in *Proceedings of the International Symposium on Computer Architecture (ISCA)*, 2024.
[40] A. Patke, D. Reddy, S. Jha, H. Qiu, C. Pinto, C. Narayanaswami, Z. T. Kalbarczyk, and R. Iyer, "Queue Management for SLO-Oriented Large Language Model Serving," in *Proceedings of the ACM Symposium on Cloud Computing (SoCC)*, 2024.
[41] D. Rein, B. L. Hou, A. C. Stickland, J. Petty, R. Y. Pang, J. Dirani, J. Michael, and S. R. Bowman, "GPQA: A Graduate-Level Google-Proof Q&A Benchmark," in *Conference on Language Modeling (COLM)*, 2024.
[42] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, B. Chen, P. Liang, C. Ré, I. Stoica, and C. Zhang, "FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2023.
[43] J. Stojkovic, C. Zhang, Í. Goiri, J. Torrellas, and E. Choukse, "DynamoLLM: Designing LLM Inference Clusters for Performance and Energy Efficiency," in *Proceedings of the International Symposium on High-Performance Computer Architecture (HPCA)*, 2025.
[44] B. Sun, Z. Huang, H. Zhao, W. Xiao, X. Zhang, Y. Li, and W. Lin, "Llumnix: Dynamic Scheduling for Large Language Model Serving," in *Proceedings of the USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.
[45] Q. Team, "QwQ-32B: Embracing the Power of Reinforcement Learning," 2025. [Online]. Available: https://qwenlm.github.io/blog/qwq-32b/
[46] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-consistency Improves Chain of Thought Reasoning in Language Models," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2023.
[47] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. V. Le, and D. Zhou, "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," in *Proceedings of the International Conference on Neural Information Processing Systems (NeurIPS)*, 2022.
[48] B. Wu, Y. Zhong, Z. Zhang, S. Liu, F. Liu, Y. Sun, G. Huang, X. Liu, and X. Jin, "Fast Distributed Inference Serving for Large Language Models," in *arxiv.org*, 2023.
[49] S. Yao, E. Nijkamp, M. Bosma, X. Zhao, T. Wu, D. Zhou, and D. Schuurmans, "ReAct: Synergizing Reasoning and Acting in Language Models," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2023.
[50] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan, "Tree of Thoughts: Deliberate Problem Solving with Large Language Models," in *Proceedings of the International Conference on Neural Information Processing Systems (NeurIPS)*, 2023.
[51] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "ORCA: A Distributed Serving System for Transformer-Based Generative Models," in *Proceedings of the USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2022.
[52] Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Ré, C. Barrett *et al.*, "H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models," 2023.
[53] Y. Zhao, D. Wu, and J. Wang, "ALISA: Accelerating Large Language Model Inference via Sparsity-Aware KV Caching," in *Proceedings of the International Symposium on Computer Architecture (ISCA)*, 2024.
[54] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang, "DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving," in *Proceedings of the USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.
[55] D. Zhou, N. Schärli, L. Hou, J. Wei, N. Scales, X. Wang, D. Schuurmans, C. Cui, O. Bousquet, Q. Le, and E. Chi, "Least-to-Most Prompting Enables Complex Reasoning in Large Language Models," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2023.